

MACHINE LEARNING (ML)

BTCSE023602

UNIT-IV

UNSUPERVISED LEARNING

Mr. Naresh A. Kamble

Asst. Professor,

Department of Computer Science and Engineering

D.Y.Patil Agriculture and Technical University, Talsande, Kolhapur

COURSE OUTCOME

After end of this unit, students will be able to

CO3: Analyze and Implement Unsupervised Learning Techniques.

CONTENTS

- **Clustering Algorithms**
- **Dimensionality Reduction**
- **Anomaly Detection**

CLUSTERING

CLUSTERING

- **Clustering** or **cluster analysis** is a machine learning technique, which groups the **unlabeled dataset**.
- It can be defined as "*A way of grouping the data points into different clusters, consisting of similar data points. The objects with the possible similarities remain in a group that has less or no similarities with another group.*"

CLUSTERING

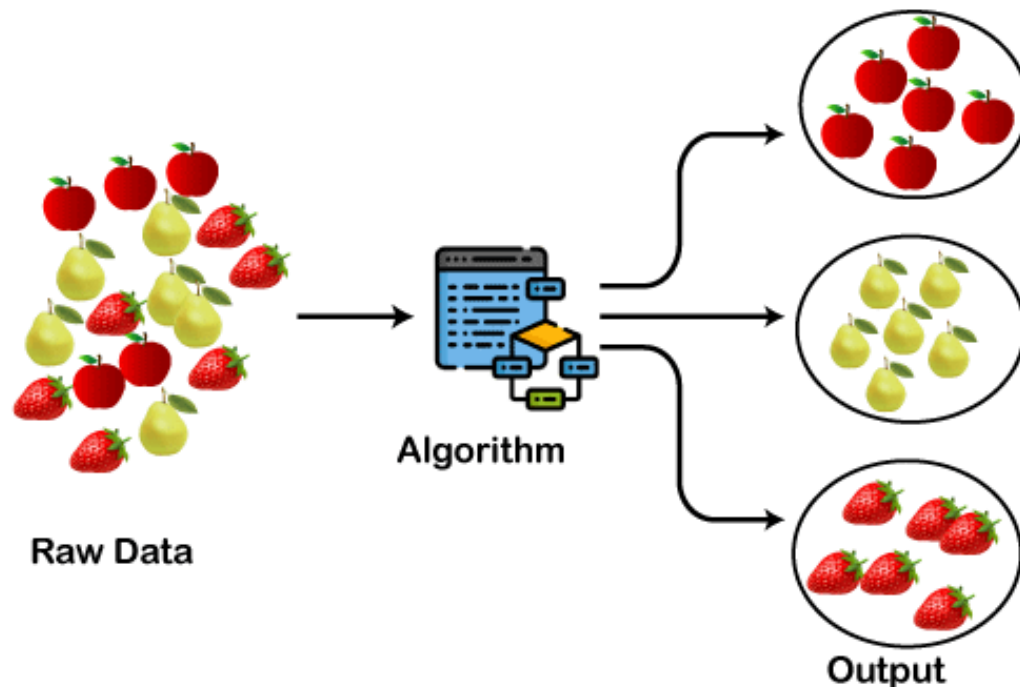
- It does it by finding some **similar patterns** in the unlabelled dataset such as **shape, size, color, behavior**, etc., and divides them as per the **presence** and **absence** of those **similar patterns**.
- The clustering technique is commonly used for **statistical data analysis**.

CLUSTERING

- **Example:** Let's understand the clustering technique with the real-world example of Mall: When we visit any shopping mall, we can observe that the things with similar usage are grouped together.
- Such as the t-shirts are grouped in one section, and trousers are at other sections, similarly, at vegetable sections, apples, bananas, Mangoes, etc., are grouped in separate sections, so that we can easily find out the things.

CLUSTERING

- The below diagram explains the working of the clustering algorithm. We can see the different fruits are divided into several groups with similar properties.



CLUSTERING

- **HARD CLUSTERING**
- *In this type of clustering, each data point belongs to a cluster completely or not.*
- For example, Let's say there are 4 data point and we have to cluster them into 2 clusters.
- So each data point will either belong to cluster 1 or cluster 2.

CLUSTERING

- HARD CLUSTERING**

Data Points	Clusters
A	C1
B	C2
C	C2
D	C1

CLUSTERING

- **SOFT CLUSTERING**
- *In this type of clustering, instead of assigning each data point into a separate cluster, a probability or likelihood of that point being that cluster is evaluated.*
- For example, Let's say there are 4 data point and we have to cluster them into 2 clusters.
- So we will be evaluating a probability of a data point belonging to both clusters. This probability is calculated for all data points.

CLUSTERING

- **SOFT CLUSTERING**

Data Points	Probability of C1	Probability of C2
A	0.91	0.09
B	0.3	0.7
C	0.17	0.83
D	1	0

CLUSTERING

- **TYPES OF CLUSTERING**
- **Partitioning Clustering**
- **Density-Based Clustering**
- **Distribution Model-Based Clustering**
- **Hierarchical Clustering**
- **Fuzzy Clustering**

CLUSTERING

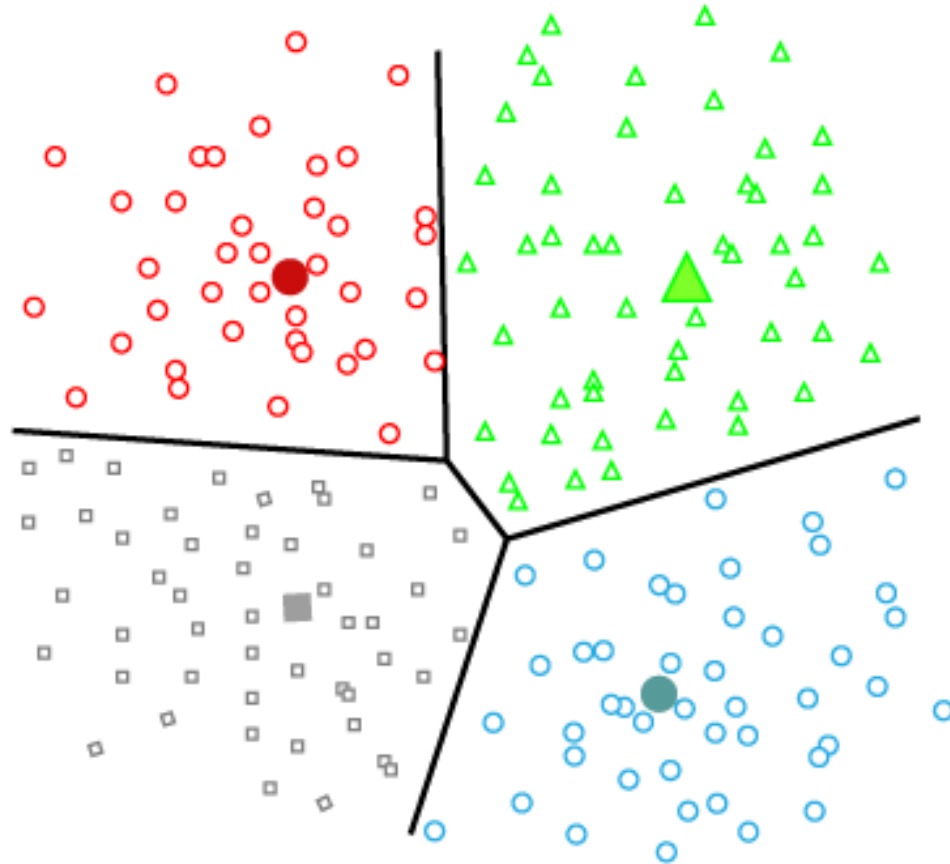
- **PARTITIONING CLUSTERING**
- It is a type of clustering that divides the data into **non-hierarchical groups**.
- It is also known as the **centroid-based method**.
- The most common example of partitioning clustering is the **K-Means Clustering algorithm**.

CLUSTERING

- **PARTITIONING CLUSTERING**
- In this type, the dataset is divided into a set of k groups, where K is used to define the number of pre-defined groups.
- The cluster center is created in such a way that the **distance between the data points of one cluster is minimum as compared to another cluster centroid.**

CLUSTERING

- PARTITIONING CLUSTERING**

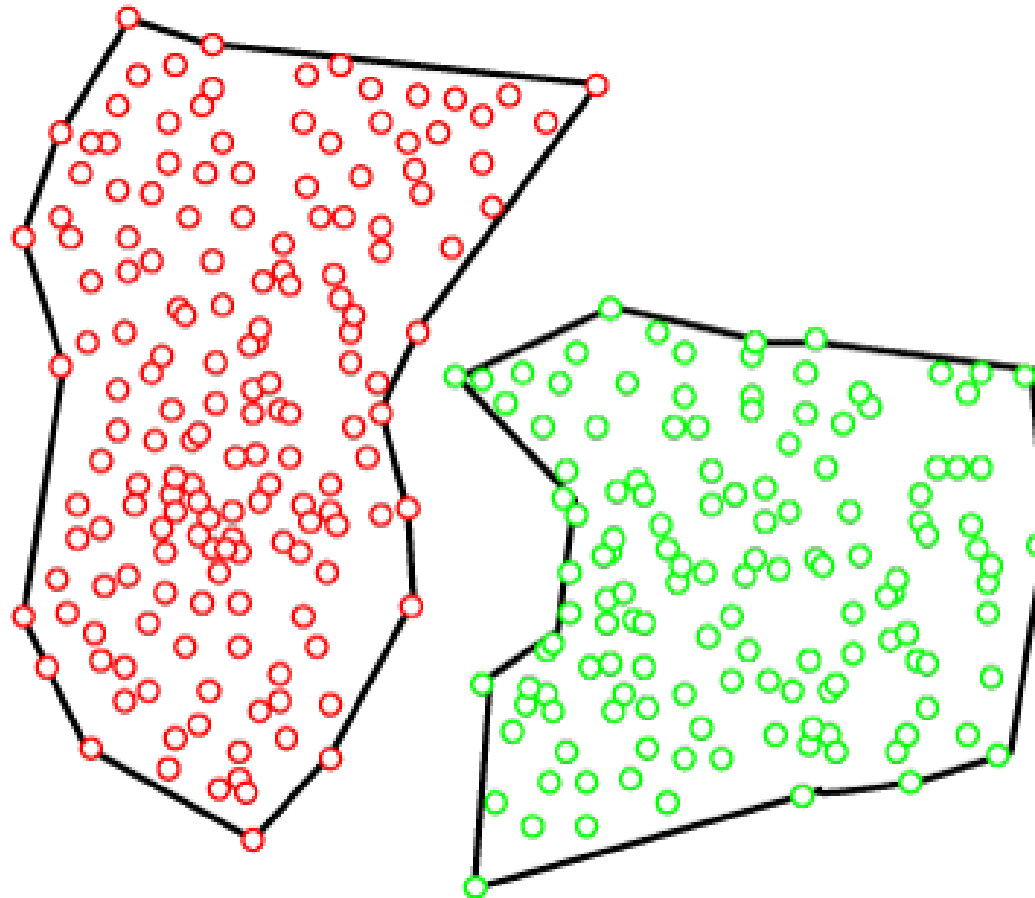


CLUSTERING

- **DENSITY BASED CLUSTERING**
- The density-based clustering method connects the **highly-dense areas** into **clusters**, and the **arbitrarily shaped distributions** are formed as long as **the dense region** can be **connected**.
- This algorithm does it by **identifying different clusters** in the **dataset** and connects the areas of **high densities** into **clusters**.

CLUSTERING

- **DENSITY BASED CLUSTERING**

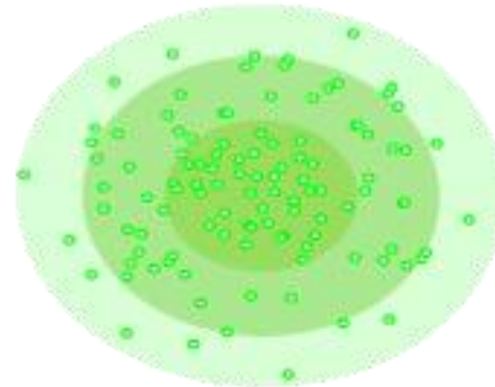
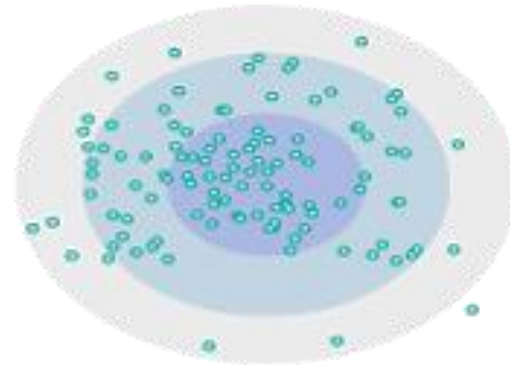
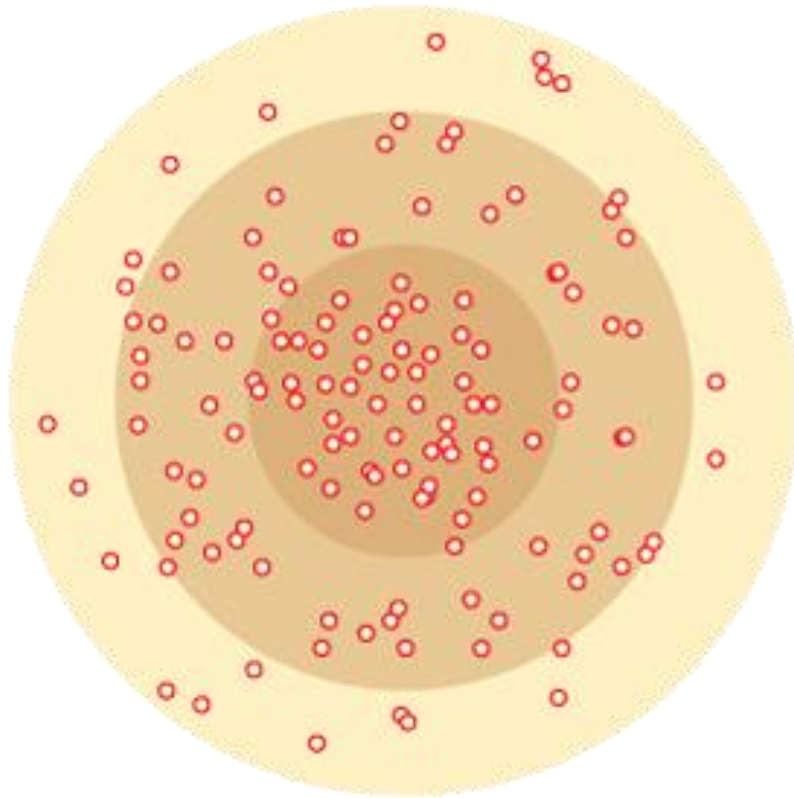


CLUSTERING

- **DISTRIBUTION MODEL-BASED CLUSTERING**
- In the **distribution model-based clustering** method, the data is divided based on the **probability** of how a dataset belongs to a **particular distribution**.

CLUSTERING

- DISTRIBUTION MODEL-BASED CLUSTERING**

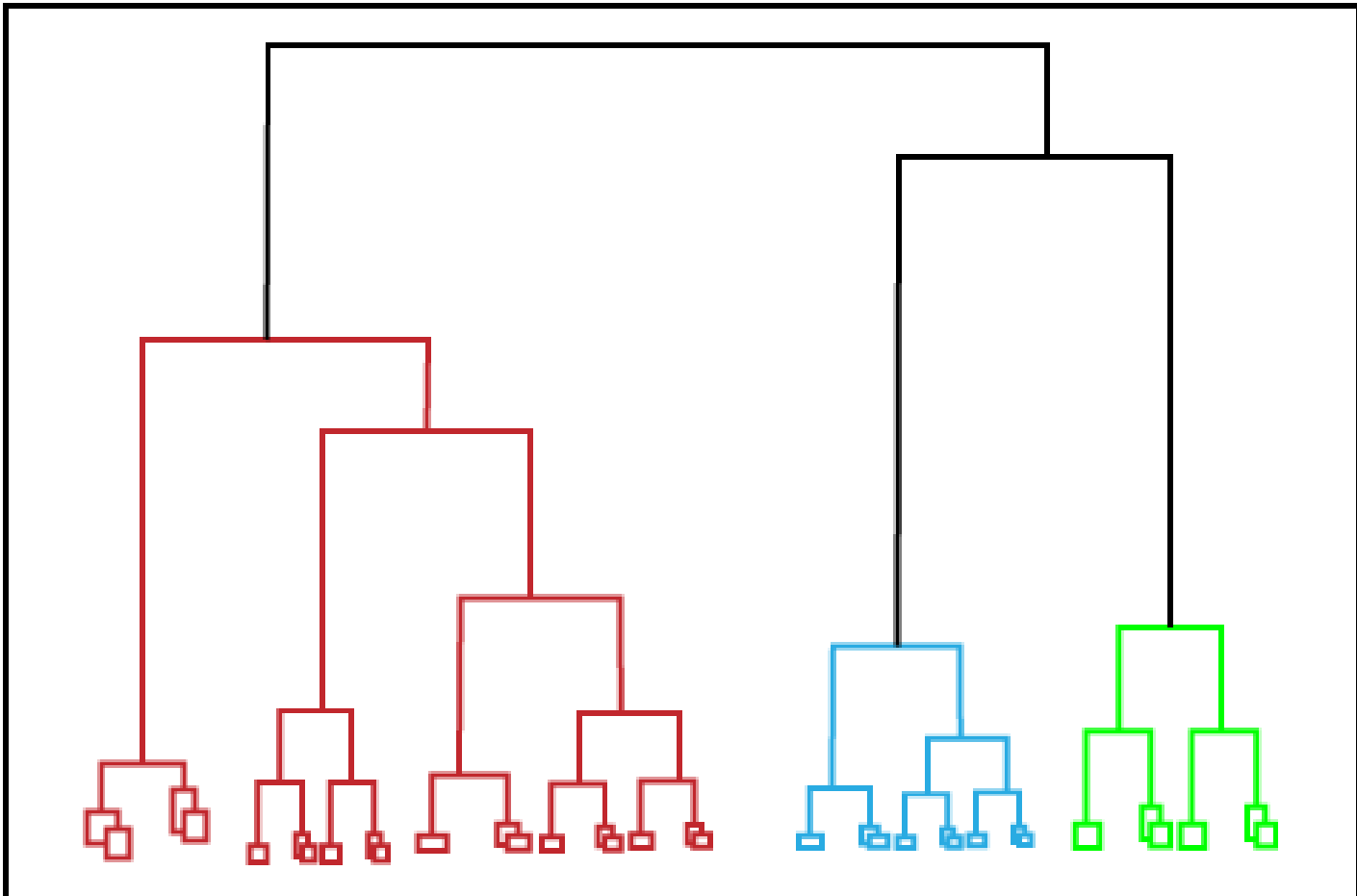


CLUSTERING

- **HIERARCHICAL CLUSTERING**
- In this technique, the dataset is divided into clusters to create a tree-like structure, which is also called a **dendrogram**.
- The observations or any number of clusters can be selected by cutting the tree at the correct level.

CLUSTERING

- **HIERARCHICAL CLUSTERING**



CLUSTERING

- **FUZZY CLUSTERING**
- **Fuzzy clustering** is a type of **soft method** in which a data object may belong to more than one group or cluster.
- Fuzzy C-means algorithm is the example of this type of clustering; it is sometimes also known as the Fuzzy k-means algorithm.

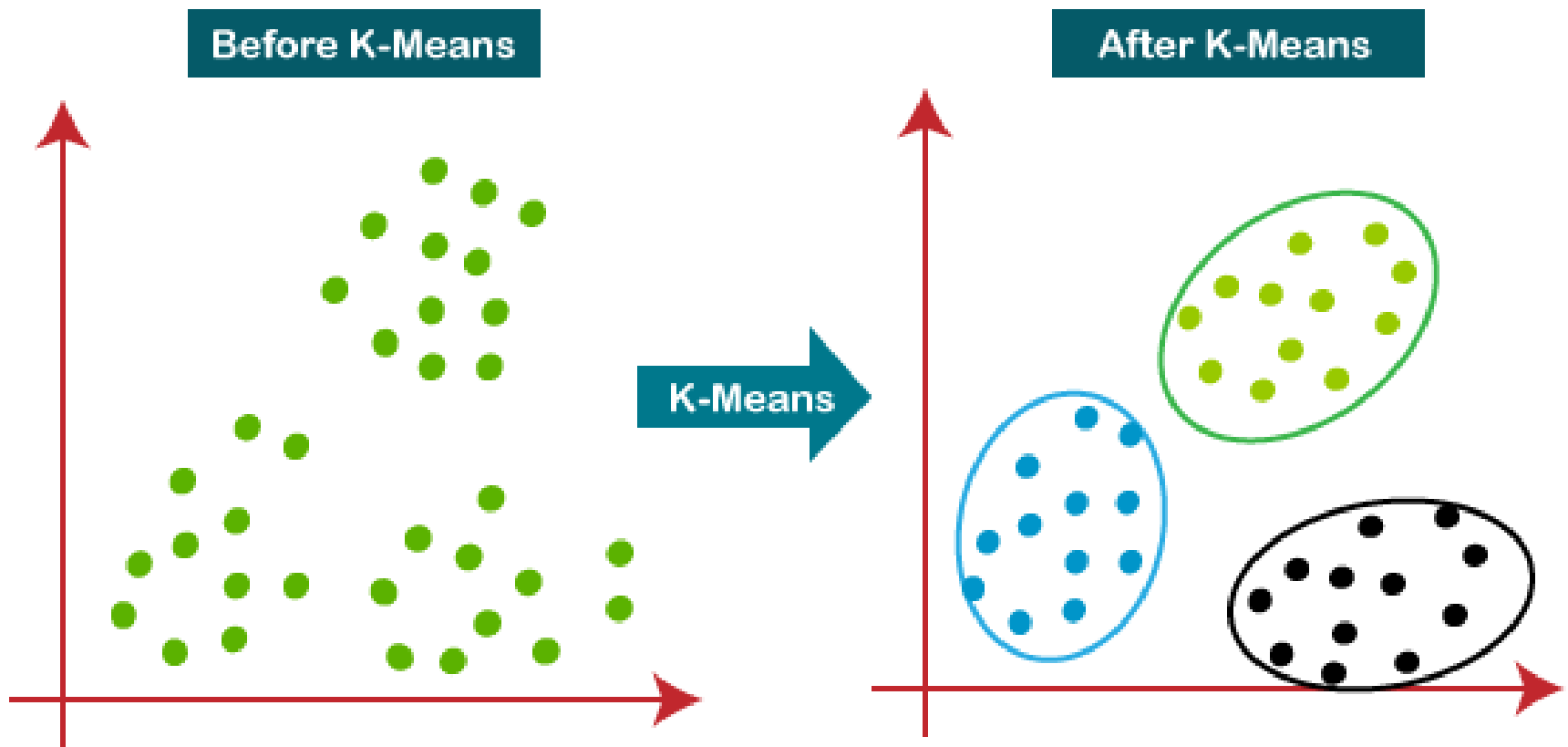
K-MEAN CLUSTERING ALGORITHM

- **INTRODUCTION**
- K-Means Clustering is an **Unsupervised Learning algorithm**, which groups the **unlabeled dataset** into different clusters.
- Here **K** defines the number of pre-defined clusters that need to be created in the process, as if **K=2**, there will be two clusters, and for **K=3**, there will be three clusters, and so on.
- It is an **iterative algorithm** that divides the unlabeled dataset into **k** different clusters in such a way that each dataset belongs only one group that has similar properties.
- It is a **centroid-based algorithm**, where each cluster is associated with a **centroid**.
- The main aim of this algorithm is to **minimize** the **sum of distances** between the **data point** and their **corresponding clusters**.

K-MEAN CLUSTERING ALGORITHM

- **INTRODUCTION**
- The k-means clustering algorithm mainly performs two tasks:
- **Determines the best value for K center points or centroids by an iterative process.**
- **Assigns each data point to its closest k-center. Those data points which are near to the particular k-center, create a cluster.**

K-MEAN CLUSTERING ALGORITHM



K-MEAN CLUSTERING ALGORITHM

- **WORKING OF K-MEANS CLUSTERING ALGORITHM**
- **Step-1:** Select the number **K** to decide the number of clusters.
- **Step-2:** Select random **K** points or **centroids**. (It can be other from the input dataset).
- **Step-3:** Assign each data point to their **closest centroid**, which will form the predefined **K clusters**.
- **Step-4:** Calculate the **variance** and place a **new centroid** of each cluster.
- **Step-5:** Repeat the **third steps**, which means reassign each data point to the new closest centroid of each cluster.
- **Step-6:** If any **reassignment** occurs, then go to step-4 else go to FINISH.
- **Step-7:** The model is ready.

K-MEAN CLUSTERING ALGORITHM

- **EXAMPLE**
- Use K Means clustering to cluster the following data into two groups.
 - Data Points: { 2, 4, 10, 12, 3, 20, 30, 11, 25 }
- The distance function used is Euclidean distance.
- Initial cluster centroid are $M1 = 4$ and $M2 = 11$.

K-MEANS EXAMPLES

K-MEAN CLUSTERING ALGORITHM

Initial Centroids:

M1: 4

M2: 11

Therefore

C1 = {2, 4, 3}

C2 = {10, 12, 20, 30, 11, 25}

C1 = {2+4+3}=9/3=**3**

C2 =

{10+12+20+30+11+25}=108/6=**18**

New Centroids:

M1: 3

M2: 18

Data Points	Distance to		Cluster	New Cluster
	M1	M2		
2	2	9	C1	
4	0	7	C1	
10	6	1	C2	
12	8	1	C2	
3	1	8	C1	
20	16	9	C2	
30	26	19	C2	
11	7	0	C2	
25	21	14	C2	

$$d(x_2, x_1) = \sqrt{(x_2 - x_1)^2}$$

K-MEAN CLUSTERING ALGORITHM

Current Centroids:

M1: 3

M2: 18

Therefore

C1 = {2, 4, 10, 3}

C2 = {12, 20, 30, 11, 25}

C1 = {2+4+10+3}=19/4=**4.75**

C2 = {12+20+30+11+25}=98/5=**19.6**

New Centroids:

M1: 4.75

M2: 19.6

Data Points	Distance to		Cluster	New Cluster
	M1	M2		
2	1	16	C1	C1
4	1	14	C1	C1
10	7	8	C2	C1
12	9	6	C2	C2
3	0	15	C1	C1
20	17	2	C2	C2
30	27	12	C2	C2
11	8	7	C2	C2
25	22	7	C2	C2

$$d(x_2, x_1) = \sqrt{(x_2 - x_1)^2}$$

K-MEAN CLUSTERING ALGORITHM

Current Centroids:

M1: 4.75

M2: 19.6

Therefore

C1 = {2, 4, 10, 11, 12, 3}

C2 = {20, 30, 25}

C1 = {2+4+10+11+12+3}=42/7=**7**

C2 = {20+30+25}=75/3=**25**

New Centroids:

M1: 7

M2: 25

Data Points	Distance to		Cluster	New Cluster
	M1	M2		
2	2.75	17.6	C1	C1
4	0.75	15.6	C1	C1
10	5.25	9.6	C1	C1
12	7.25	7.6	C2	C1
3	1.75	16.6	C1	C1
20	15.25	0.4	C2	C2
30	25.25	10.4	C2	C2
11	6.25	8.6	C2	C1
25	20.25	5.4	C2	C2

$$d(x_2, x_1) = \sqrt{(x_2 - x_1)^2}$$

K-MEAN CLUSTERING ALGORITHM

Current Centroids:

M1: 7

M2: 25

Final Cluster are:

C1= {2, 4, 10, 11, 12, 3}

C2= {20, 30, 25}

Data Points	Distance to		Cluster	New Cluster
	M1	M2		
2	5	23	C1	C1
4	3	21	C1	C1
10	3	15	C1	C1
12	5	13	C1	C1
3	4	22	C1	C1
20	13	5	C2	C2
30	23	5	C2	C2
11	4	14	C1	C1
25	18	0	C2	C2

$$d(x_2, x_1) = \sqrt{(x_2 - x_1)^2}$$

K-MEANS EXAMPLES TO BE SOLVED BY STUDENTS

K-MEAN CLUSTERING ALGORITHM

- **EXAMPLE 1**
- Data Points : { 3, 5, 11, 13, 4, 21, 31, 11, 25}
 - Initial Centroids: M1-5 , M2-11

K-MEAN CLUSTERING ALGORITHM

- Solution**

Initial Centroids

M1: 5

M2: 11

Clusters

C1: {3,5,4}

C2: {11,13,21,31,11,25}

$C1 = \{3+5+4\}=12/3=4$

C2 =

$\{11+13+21+31+11+25\}=112/6=18.67$

67

New Centroids

M1: 4

M2: 18.67

Data Points	Distance to		Cluster	New Cluster
	M1	M2		
3	2	8	C1	
5	0	6	C1	
11	6	0	C2	
13	8	2	C2	
4	1	7	C1	
21	16	10	C2	
31	26	20	C2	
11	6	0	C2	
25	20	14	C2	

$$d(x_2, x_1) = \sqrt{(x_2 - x_1)^2}$$

K-MEAN CLUSTERING ALGORITHM

- Solution**

Current Centroids

M1: 4

M2: 18.67

Clusters

C1: {3,5,11,4,11}

C2: {13,21,31,25}

C1 = {3+5+11+4+11}=34/5=**6.8**

C2 = {13+21+31+25}=90/4=**22.5**

New Centroids

M1: 6.8

M2: 22.5

Data Points	Distance to		Cluster	New Cluster
	M1	M2		
3	1	15.67	C1	C1
5	1	13.67	C1	C1
11	7	7.67	C2	C1
13	9	5.67	C2	C2
4	0	14.67	C1	C1
21	17	2.33	C2	C2
31	27	12.33	C2	C2
11	7	7.67	C2	C1
25	21	6.33	C2	C2

$$d(x_2, x_1) = \sqrt{(x_2 - x_1)^2}$$

K-MEAN CLUSTERING ALGORITHM

- Solution**

Current Centroids

M1: 6.8

M2: 22.5

Clusters

C1: {3,5,11,13,4,11}

C2: {21,31,25}

C1 =

{3+5+11+13+4+11}=47/6=**7.83**

C2 = {21+31+25}=77/3=**25.67**

New Centroids

M1: 7.83

M2: 25.67

Data Points	Distance to		Cluster	New Cluster
	M1	M2		
3	3.8	19.5	C1	C1
5	1.8	17.5	C1	C1
11	4.2	11.5	C1	C1
13	6.2	9.5	C2	C1
4	2.8	18.5	C1	C1
21	14.2	1.5	C2	C2
31	24.2	8.5	C2	C2
11	4.2	11.5	C1	C1
25	18.2	2.5	C2	C2

$$d(x_2, x_1) = \sqrt{(x_2 - x_1)^2}$$

K-MEAN CLUSTERING ALGORITHM

- Solution**

Current Centroids

M1: 7.83

M2: 25.67

Clusters

C1: {3,5,11,13,4,11}

C2: {21,31,25}

Data Points	Distance to		Cluster	New Cluster
	M1	M2		
3	4.83	22.67	C1	C1
5	2.83	20.67	C1	C1
11	3.17	14.67	C1	C1
13	5.17	12.67	C1	C1
4	3.83	21.67	C1	C1
21	13.17	4.67	C2	C2
31	23.17	5.33	C2	C2
11	3.17	14.67	C1	C1
25	17.17	0.67	C2	C2

$$d(x_2, x_1) = \sqrt{(x_2 - x_1)^2}$$

K-MEAN CLUSTERING ALGORITHM

- **EXAMPLE 2**
- Data Points : { 4, 5, 12, 11, 3, 20, 28, 10, 24}
 - Initial Centroids: M1-4 , M2-10

K-MEAN CLUSTERING ALGORITHM

- Solution**

Initial Centroids

M1: 4

M2: 10

Clusters

C1: {4,5,3}

C2: {12,11,20,28,10,24}

New Centroids

M1: 4

M2: 17.5

Data Points	Distance to		Cluster	New Cluster
	M1	M2		
4	0	6	C1	
5	1	5	C1	
12	8	2	C2	
11	7	1	C2	
3	1	7	C1	
20	16	10	C2	
28	24	18	C2	
10	6	0	C2	
24	20	14	C2	

$$d(x_2, x_1) = \sqrt{(x_2 - x_1)^2}$$

K-MEAN CLUSTERING ALGORITHM

- Solution**

Current Centroids

M1: 4

M2: 17.5

Clusters

C1: {4,5,3,10}

C2: {12,11,20,28,24}

New Centroids

M1: 5.5

M2: 19

Data Points	Distance to		Cluster	New Cluster
	M1	M2		
4	0	13.5	C1	C1
5	1	12.5	C1	C1
12	8	5.5	C2	C2
11	7	6.5	C2	C2
3	1	14.5	C1	C1
20	16	2.5	C2	C2
28	24	10.5	C2	C2
10	6	7.5	C2	C1
24	20	5	C2	C2

$$d(x_2, x_1) = \sqrt{(x_2 - x_1)^2}$$

K-MEAN CLUSTERING ALGORITHM

- Solution**

Current Centroids

M1: 5.5

M2: 19

Clusters

C1: {4,5,12,11,3,10}

C2: {20,28,24}

New Centroids

M1: 6.6

M2: 24

Data Points	Distance to		Cluster	New Cluster
	M1	M2		
4	1.5	15	C1	C1
5	0.5	14	C1	C1
12	6.5	7	C1	C1
11	5.5	8	C1	C1
3	2.5	16	C1	C1
20	14.5	1	C2	C2
28	22.5	9	C2	C2
10	4.5	9	C1	C1
24	18.5	5	C2	C2

$$d(x_2, x_1) = \sqrt{(x_2 - x_1)^2}$$

K-MEAN CLUSTERING ALGORITHM

- Solution**

Current Centroids

M1: 6.6

M2: 24

Clusters

C1: {4,5,12,11,3,10}

C2: {20,28,24}

New Centroids

M1: 6.6

M2: 24

Data Points	Distance to		Cluster	New Cluster
	M1	M2		
4	2.6	20	C1	C1
5	1.6	19	C1	C1
12	5.4	12	C1	C1
11	4.4	13	C1	C1
3	3.6	21	C1	C1
20	13.4	4	C2	C2
28	21.4	4	C2	C2
10	3.4	14	C1	C1
24	17.4	0	C2	C2

$$d(x_2, x_1) = \sqrt{(x_2 - x_1)^2}$$

HIERARCHICAL CLUSTERING ALGORITHM (HCA)

HIERARCHICAL CLUSTERING ALGORITHM

- **INTRODUCTION**
- Hierarchical clustering is another unsupervised machine learning algorithm, which is used to group the unlabeled datasets into a cluster and also known as hierarchical cluster analysis or HCA.
- In this algorithm, we develop the hierarchy of clusters in the form of a tree, and this tree-shaped structure is known as the dendrogram.

HIERARCHICAL CLUSTERING ALGORITHM

- **INTRODUCTION**

- The hierarchical clustering technique has two approaches:
- **Agglomerative:** Agglomerative is a **bottom-up** approach, in which the algorithm starts with taking all **data points** as **single clusters** and **merging** them until **one cluster** is left.
- **Divisive:** Divisive algorithm is the **reverse** of the **agglomerative algorithm** as it is a **top-down** approach.

HIERARCHICAL CLUSTERING ALGORITHM

- NEED OF HCA
- Problems with K-Means Algorithm
- Predetermined number of clusters
- It always tries to create the clusters of the same size.

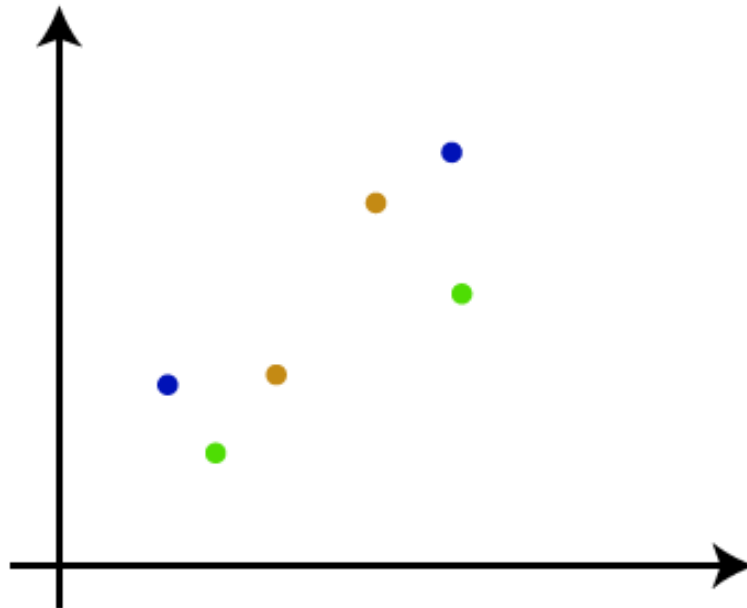
AGGLOMERATIVE HIERARCHICAL CLUSTERING

HIERARCHICAL CLUSTERING ALGORITHM

- To group the datasets into **clusters**, it follows the **bottom-up** approach.
- It means, this algorithm considers each **dataset** as a **single cluster** at the **beginning**, and then start combining the **closest pair of clusters** together.
- It does this until all the **clusters** are **merged** into a **single cluster** that contains all the **datasets**.
- This **hierarchy** of **clusters** is represented in the form of the **dendrogram**.

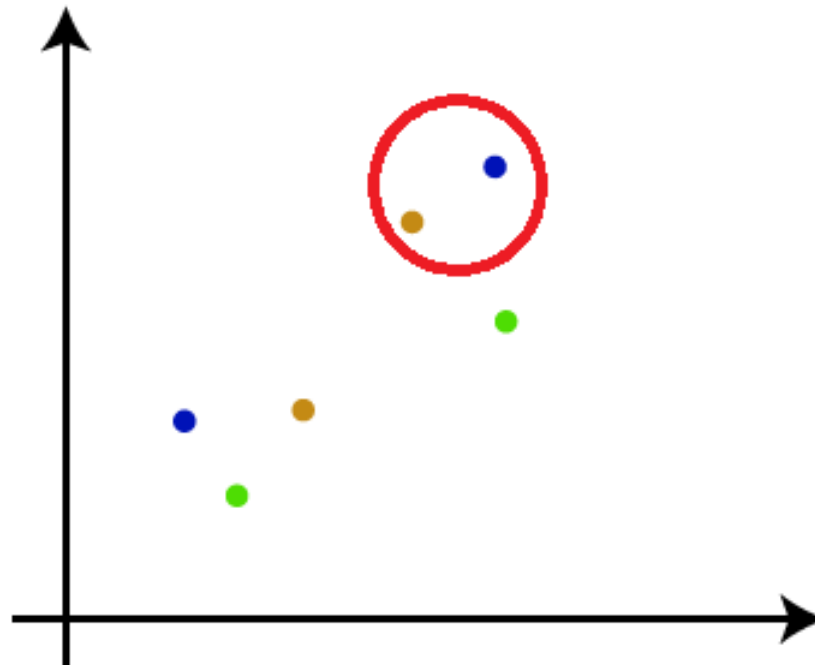
HIERARCHICAL CLUSTERING ALGORITHM

- **WORKING OF AGGLOMERATIVE HCA**
- **Step-1:** Create each data point as a **single cluster**. Let's say there are **N data points**, so the **number of clusters** will also be **N**.



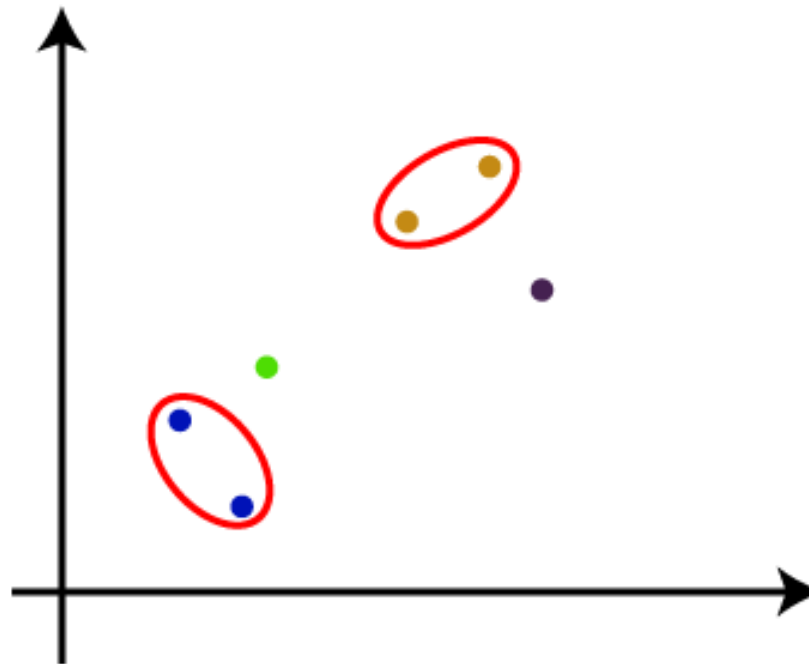
HIERARCHICAL CLUSTERING ALGORITHM

- **WORKING OF AGGLOMERATIVE HCA**
- **Step-2:** Take **two closest data points or clusters** and **merge** them to form **one cluster**. So, there will now be **N-1 clusters**.



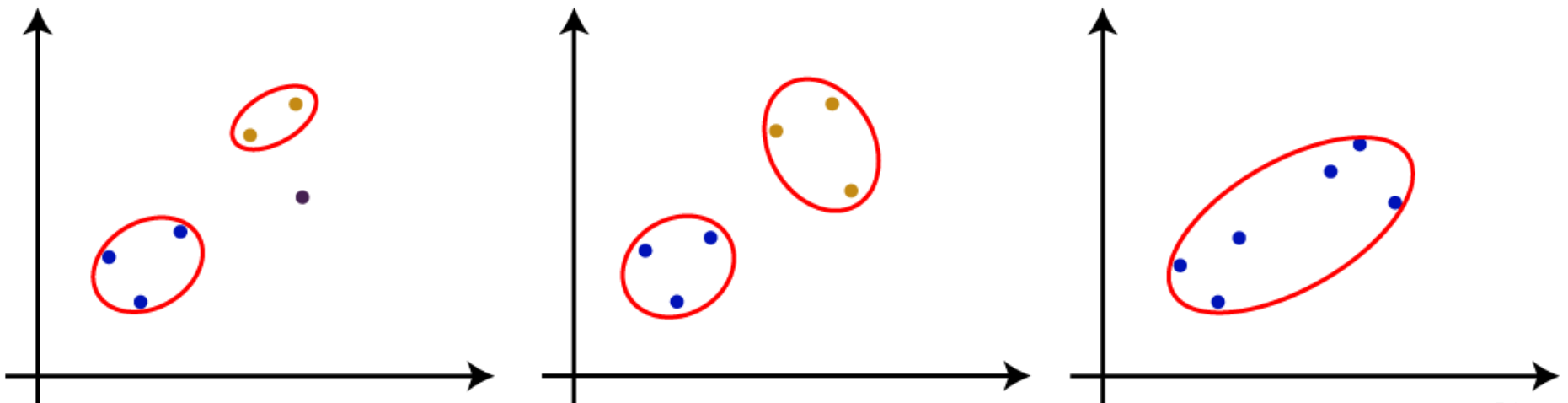
HIERARCHICAL CLUSTERING ALGORITHM

- **WORKING OF AGGLOMERATIVE HCA**
- **Step-3:** Again, take the **two closest clusters** and **merge** them **together** to form **one cluster**. There will be **N-2 clusters**.



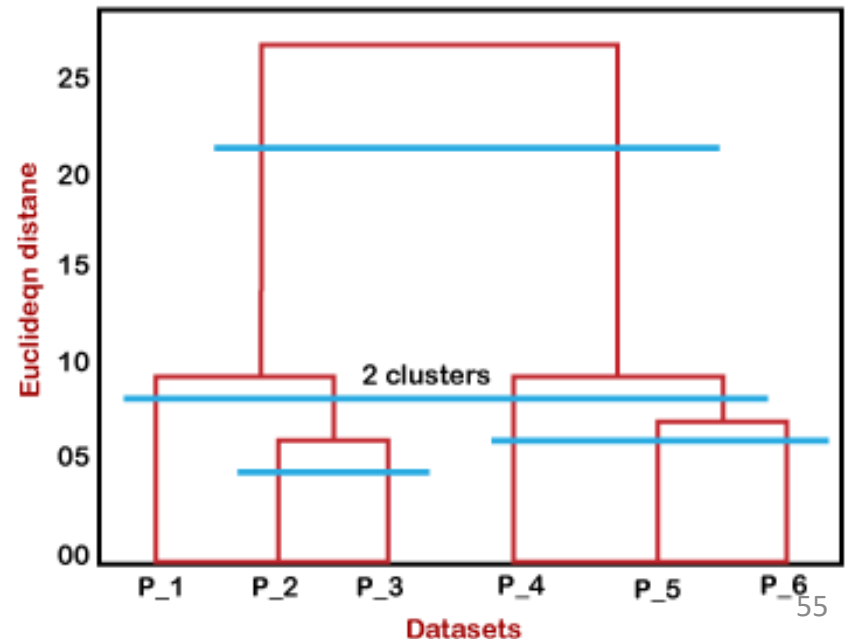
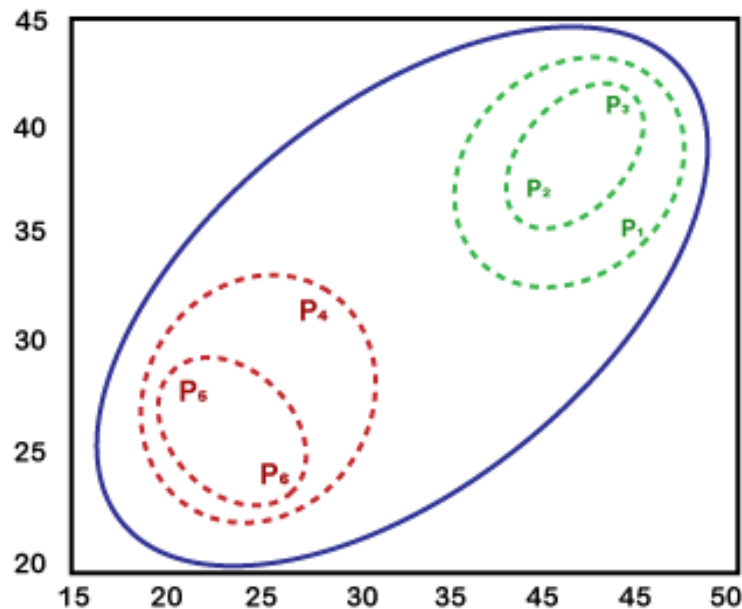
HIERARCHICAL CLUSTERING ALGORITHM

- **WORKING OF AGGLOMERATIVE HCA**
- **Step-4:** Repeat **Step 3** until only **one cluster** left. So, we will get the following **clusters**. Consider the below images:



HIERARCHICAL CLUSTERING ALGORITHM

- **WORKING OF AGGLOMERATIVE HCA**
- **Step-5:** Once all the **clusters** are **combined** into **one big cluster**, develop the **dendrogram** to **divide** the **clusters** as per the **problem**.



AGGLOMERATIVE HIERARCHICAL CLUSTERING EXAMPLE

HIERARCHICAL CLUSTERING ALGORITHM

- **EXAMPLE**

- Consider the following set of 6 one dimensional data points:
- 18, 22, 25, 42, 27, 43
- Apply the **agglomerative hierarchical clustering** algorithm to build the hierarchical clustering **dendogram**.
- Merge the clusters using **Min distance** and update the proximity matrix accordingly.
- Clearly show the **proximity matrix** corresponding to each iteration of the algorithm.

HIERARCHICAL CLUSTERING ALGORITHM

- **STEP-1:** Create a matrix of given datapoints and find minimum distance.

	18	22	25	27	42	43
18	0	4	7	9	24	25
22	4	0	3	5	20	21
25	7	3	0	2	17	18
27	9	5	2	0	15	16
42	24	20	17	15	0	1
43	25	21	18	16	1	0

HIERARCHICAL CLUSTERING ALGORITHM

- **STEP-2:** Merge the minimum distance datapoints into single cluster
- Cluster 1 – (42,43)

	18	22	25	27	42	43
18	0	4	7	9	24	25
22	4	0	3	5	20	21
25	7	3	0	2	17	18
27	9	5	2	0	15	16
42	24	20	17	15	0	1
43	25	21	18	16	1	0

	18	22	25	27	(42,43)
18	0	4	7	9	24
22	4	0	3	5	20
25	7	3	0	2	17
27	9	5	2	0	15
(42,43)	24	20	17	15	0

HIERARCHICAL CLUSTERING ALGORITHM

- **STEP-3:** Repeat **step 2** and merge the minimum distance datapoints into single cluster

	18	22	25	27	(42,43)
18	0	4	7	9	24
22	4	0	3	5	20
25	7	3	0	2	17
27	9	5	2	0	15
(42,43)	24	20	17	15	0

HIERARCHICAL CLUSTERING ALGORITHM

- **STEP-3:** Repeat **step 2** and merge the minimum distance datapoints into single cluster
- Cluster 1 & 2 – [(42,43),(25,27)]

	18	22	25	27	(42,43)
18	0	4	7	9	24
22	4	0	3	5	20
25	7	3	0	2	17
27	9	5	2	0	15
(42,43)	24	20	17	15	0

	18	22	(25,27)	(42,43)
18	0	4	7	24
22	4	0	3	20
(25,27)	7	3	0	17
(42,43)	24	20	17	0

HIERARCHICAL CLUSTERING ALGORITHM

- **STEP-4:** Repeat **step 2** and merge the minimum distance datapoints into single cluster

	18	22	(25,27)	(42,43)
18	0	4	7	24
22	4	0	3	20
(25,27)	7	3	0	17
(42,43)	24	20	17	0

HIERARCHICAL CLUSTERING ALGORITHM

- **STEP-4:** Repeat **step 2** and merge the minimum distance datapoints into single cluster
 - Cluster 1,2 & 3 – [(42,43),((25,27),22)]

	18	22	(25,27)	(42,43)
18	0	4	7	24
22	4	0	3	20
(25,27)	7	3	0	17
(42,43)	24	20	17	0

	18	(25,27, 22)	(42,43)
18	0	4	24
(25,27, 22)	4	0	20
(42,43)	24	20	0

HIERARCHICAL CLUSTERING ALGORITHM

- **STEP-5:** Repeat **step 2** and merge the minimum distance datapoints into single cluster

	18	(25,27,22)	(42,43)
18	0	4	24
(25,27,22)	4	0	20
(42,43)	24	20	0

HIERARCHICAL CLUSTERING ALGORITHM

- **STEP-5:** Repeat **step 2** and merge the minimum distance datapoints into single cluster

	18	(25,27,22)	(42,43)
18	0	4	24
(25,27,22)	4	0	20
(42,43)	24	20	0

Cluster 1,2,3 & 4 –
 [(42,43),(((25,27),22),18)]

	(25,27,22,18)	(42,43)
(25,27,22,18)	0	24
(42,43)	24	0

HIERARCHICAL CLUSTERING ALGORITHM

- **STEP-5:** Repeat **step 2** and merge the minimum distance datapoints into single cluster

	(25,27,22,18)	(42,43)
(25,27,22,18)	0	24
(42,43)	24	0

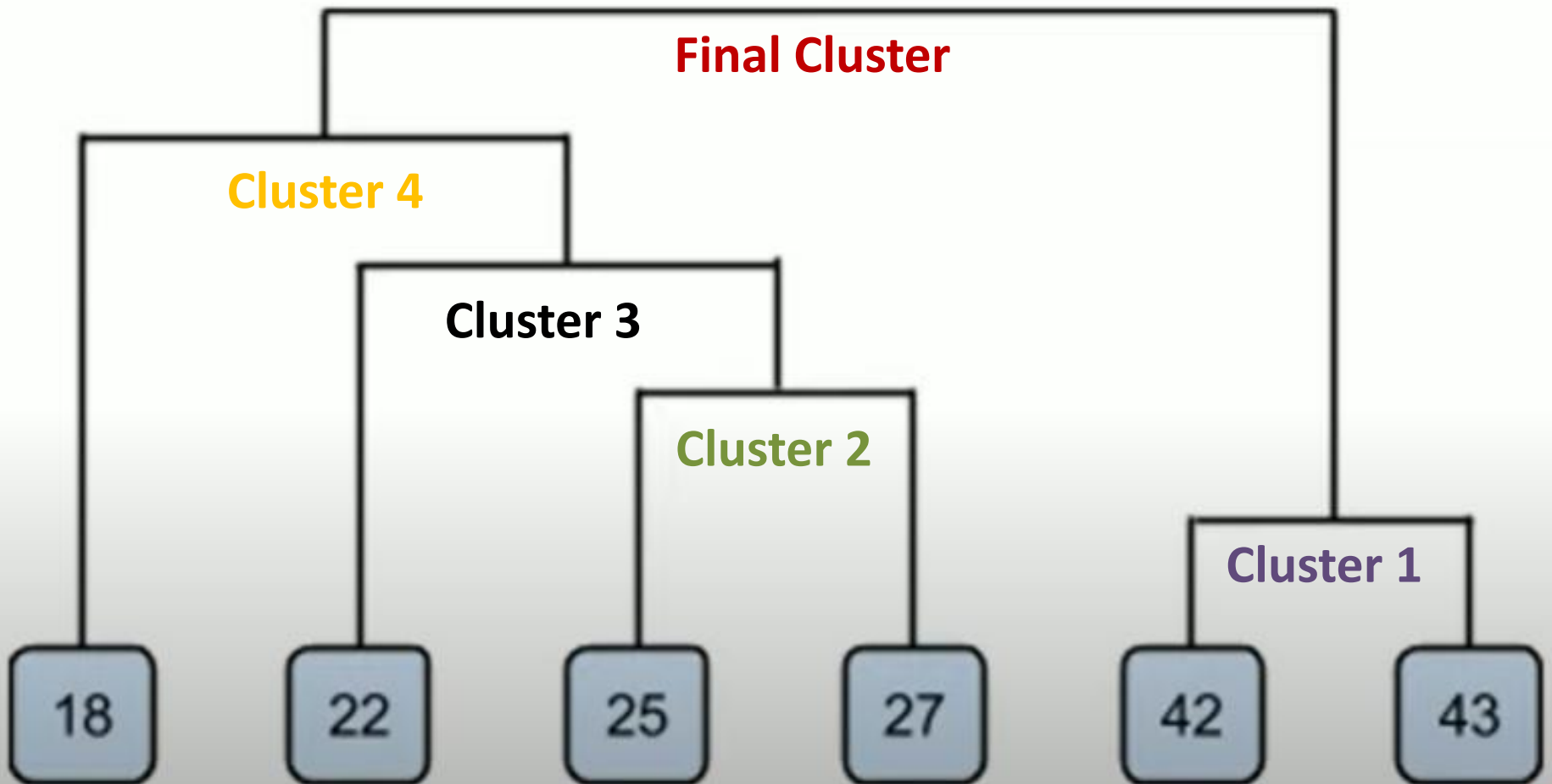
HIERARCHICAL CLUSTERING ALGORITHM

- **STEP-6:** Now convert the final clusters into *dendrogram*
 - Cluster 1,2,3 & 4 – [(42,43),(((25,27),22),18)]

	(25,27,22,18,42,43)
(25,27,22,18,42,43)	0

HIERARCHICAL CLUSTERING ALGORITHM

- DENDROGRAM - $[(42,43),(((25,27),22),18)]$



AGGLOMERATIVE HIERARCHICAL CLUSTERING EXAMPLE FOR STUDENTS

HIERARCHICAL CLUSTERING ALGORITHM

- **EXAMPLE**

- Consider the following set of 6 one dimensional data points:

- 20,24,27,44,29,45

- Apply the **agglomerative hierarchical clustering** algorithm to build the hierarchical clustering **dendogram**.
- Merge the clusters using **Min distance** and update the proximity matrix accordingly.
- Clearly show the **proximity matrix** corresponding to each iteration of the algorithm.

HIERARCHICAL CLUSTERING ALGORITHM

- **STEP-1:** Create a matrix of given datapoints and find minimum distance.

	20	24	27	29	44	45
20	0	4	7	9	24	25
24	4	0	3	5	20	21
27	7	3	0	2	17	18
29	9	5	2	0	15	16
44	24	20	17	15	0	1
45	25	21	18	16	1	0

DBSCAN ALGORITHM

DBSCAN ALGORITHM

- **INTRODUCTION**
- **DBSCAN**, which stands for *Density-Based Spatial Clustering of Applications with Noise*, is a powerful clustering algorithm that **groups points** that are **closely packed together** in **data space**.
- Unlike some other clustering algorithms, **DBSCAN** doesn't require you to specify the number of **clusters** beforehand.
- The algorithm works by defining **clusters** as **dense regions** separated by **regions of lower density**.

DBSCAN ALGORITHM

- **KEY CONCEPTS**
- **Core Points:** These are points that have at **least** a **minimum** number of **other points (MinPts)** within a specified **distance** (ϵ or epsilon).
- **Border Points:** These are points that are **within** the ϵ **distance** of a **core point** but don't have **MinPts neighbors** themselves.
- **Noise Points:** These are points that are **neither core points** nor **border points**. They're not **close** enough to **any cluster** to be included.

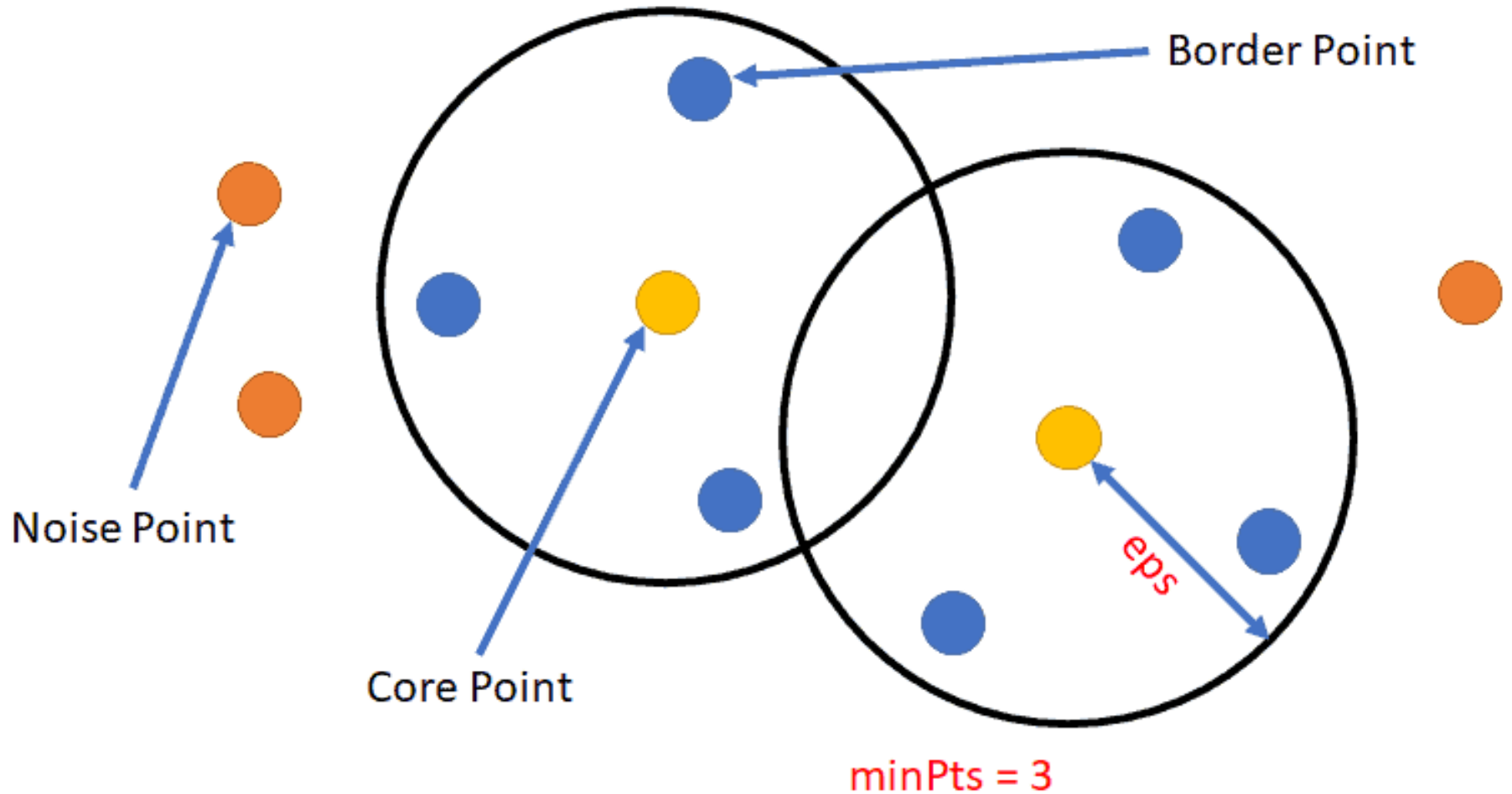
DBSCAN ALGORITHM



DBSCAN ALGORITHM

- **PARAMETERS IN DBSCAN**
- **ϵ (epsilon):** The **maximum distance** between **two points** for them to be considered as **neighbors**.
- **MinPts:** The **minimum** number of **points** required to form a **dense region**.

DBSCAN ALGORITHM



DBSCAN ALGORITHM

- **WORKING OF DBSCAN**
- **STEP-1: Parameter Selection**
 - **Choose ϵ (epsilon):** The **maximum distance** between **two points** for them to be considered as **neighbors**.
 - **Choose MinPts:** The **minimum** number of **points** required to form a **dense region**.
- **STEP-2: Select a Starting Point**
 - The algorithm starts with an **arbitrary unvisited point** in the **dataset**.

DBSCAN ALGORITHM

- **STEP-3: Examine the Neighborhood**
 - It **retrieves all points** within the **ϵ distance** of the **starting point**.
 - If the **number of neighboring points** is **less** than **MinPts**, the point is **labeled as noise** (for now).
 - If there are at **least MinPts points** within **ϵ distance**, the point is marked as a **core point**, and a **new cluster** is formed.

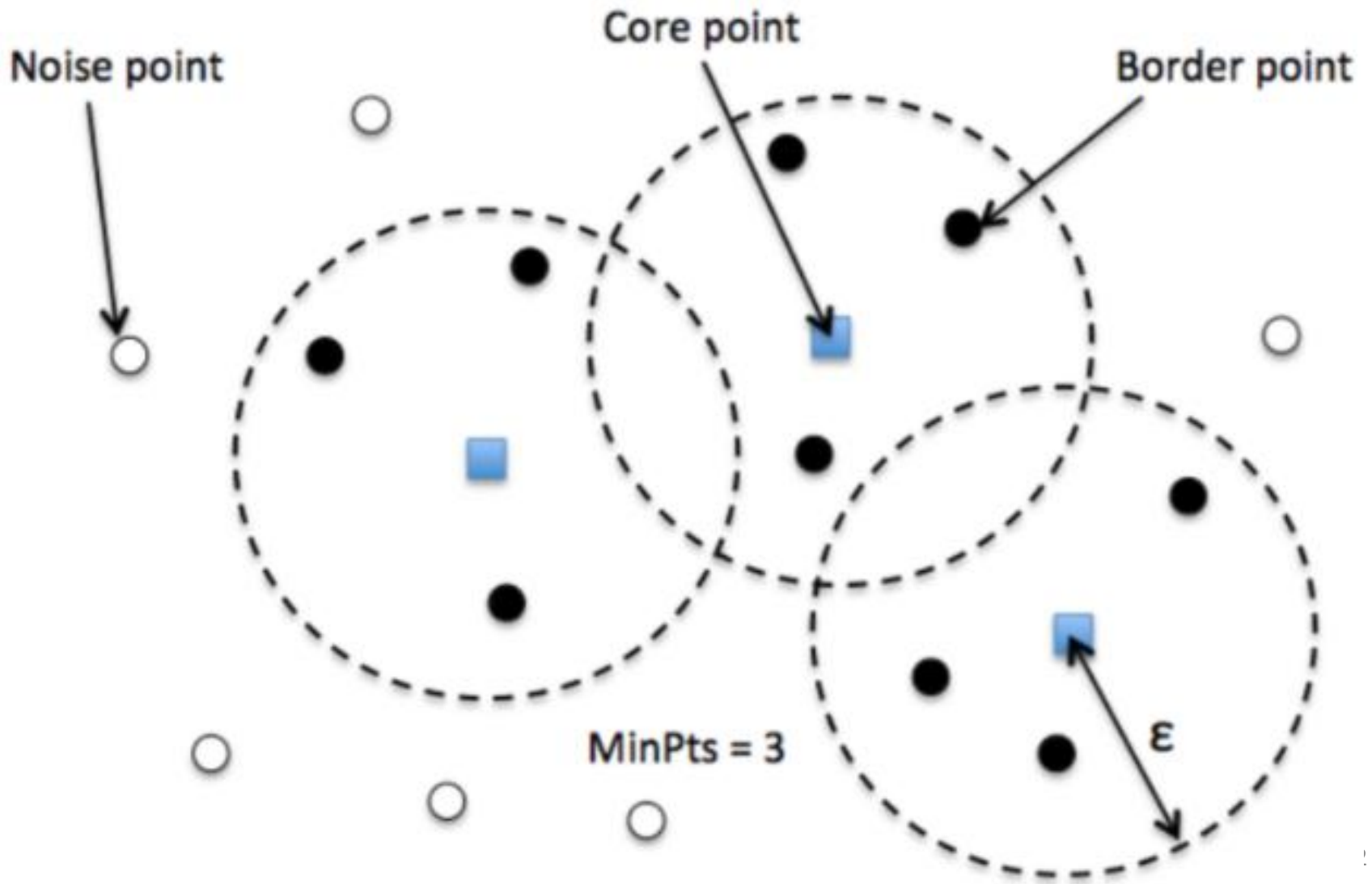
DBSCAN ALGORITHM

- **STEP-4: Expand the Cluster**
 - All the **neighbors** of the **core point** are added to the **cluster**.
 - For each of these **neighbors**:
 - If it's a **core point**, its **neighbors** are added to the **cluster recursively**.
 - If it's **not a core point**, it's **marked** as a **border point**, and the **expansion stops**.

DBSCAN ALGORITHM

- **STEP-5: Repeat the Process**
 - The algorithm moves to the **next unvisited point** in the **dataset**.
 - **Steps 3-4** are **repeated** until all **points** have been **visited**.
- **STEP-6: Finalize Clusters**
 - After **all points** have been **processed**, the algorithm **identifies** all **clusters**.
 - Points initially **labeled** as **noise** might now be **border points** if they're within ϵ **distance** of a **core point**.
- **STEP-7: Handling Noise**
 - Any points not belonging to any **cluster** remain **classified** as **noise**.

DBSCAN ALGORITHM



DBSCAN ALGORITHM EXAMPLE

DBSCAN ALGORITHM

- Apply the DBSCAN algorithm to the given data points and
- Create the clusters with
- $\text{minPts} = 4$ and
- $\text{epsilon} (\epsilon) = 1.9$.

Data Points:

P1: (3, 7)

P2: (4, 6)

P3: (5, 5)

P4: (6, 4)

P5: (7, 3)

P6: (6, 2)

P7: (7, 2)

P8: (8, 4)

P9: (3, 3)

P10: (2, 6)

P11: (3, 5)

P12: (2, 4)

DBSCAN ALGORITHM

- Use Eucladian distance and calculate the distance between each points.

$$\textit{Distance}(A(x_1, y_1), B(x_2, y_2)) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

DBSCAN ALGORITHM

- **STEP-1:** Create proximity matrix to calculate min distance between each data point.
- **MIN – 4 points** and **epsilon(€) = 1.9** $Distance(A(x_1, y_1), B(x_2, y_2)) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$

POINT	X1	Y1	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	P11	P12
P1	3	7	0											
P2	4	6	1.41	0										
P3	5	5	2.83	1.41	0									
P4	6	4	4.24	2.83	1.41	0								
P5	7	3	5.66	4.24	2.83	1.41	0							
P6	6	2	5.83	4.47	3.16	2.00	1.41	0						
P7	7	2	6.40	5.00	3.61	2.24	1.00	1.00	0					
P8	8	4	5.83	4.47	3.16	2.00	1.41	2.83	2.24	0				
P9	3	3	4.00	3.16	2.83	3.16	4.00	3.16	4.12	5.10	0			
P10	2	6	1.41	2.00	3.16	4.47	5.83	5.66	6.40	6.32	3.16	0		
P11	3	5	2.00	1.41	2.00	3.16	4.47	4.24	5.00	5.10	2.00	1.41	0	
P12	2	4	3.16	2.83	3.16	4.00	5.10	4.47	5.39	6.00	1.41	2.00	1.41	86 0

DBSCAN ALGORITHM

- **STEP-2:** Find minimum distance of each data point with other data point
- **MIN – 4 points** and **epsilon(€) = 1.9**

POINT	X1	Y1	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	P11	P12
P1	3	7	0											
P2	4	6	1.41	0										
P3	5	5	2.83	1.41	0									
P4	6	4	4.24	2.83	1.41	0								
P5	7	3	5.66	4.24	2.83	1.41	0							
P6	6	2	5.83	4.47	3.16	2.00	1.41	0						
P7	7	2	6.40	5.00	3.61	2.24	1.00	1.00	0					
P8	8	4	5.83	4.47	3.16	2.00	1.41	2.83	2.24	0				
P9	3	3	4.00	3.16	2.83	3.16	4.00	3.16	4.12	5.10	0			
P10	2	6	1.41	2.00	3.16	4.47	5.83	5.66	6.40	6.32	3.16	0		
P11	3	5	2.00	1.41	2.00	3.16	4.47	4.24	5.00	5.10	2.00	1.41	0	
P12	2	4	3.16	2.83	3.16	4.00	5.10	4.47	5.39	6.00	1.41	2.00	1.41	0

POINT	HORIZONTAL		VERTICAL		
P1			P2	P10	
P2	P1		P3	P11	
P3	P2		P4		
P4	P3		P5		
P5	P4		P6	P7	P8
P6	P5		P7		
P7	P5	P6			
P8	P5				
P9			P12		
P10	P1		P11		
P11	P2	P10	P12		
P12	P9	P11			

DBSCAN ALGORITHM

- **STEP-3:** Identify CORE, BORDER and NOISE data points.
- **MIN – 4 points** and **epsilon(€) = 1.9**

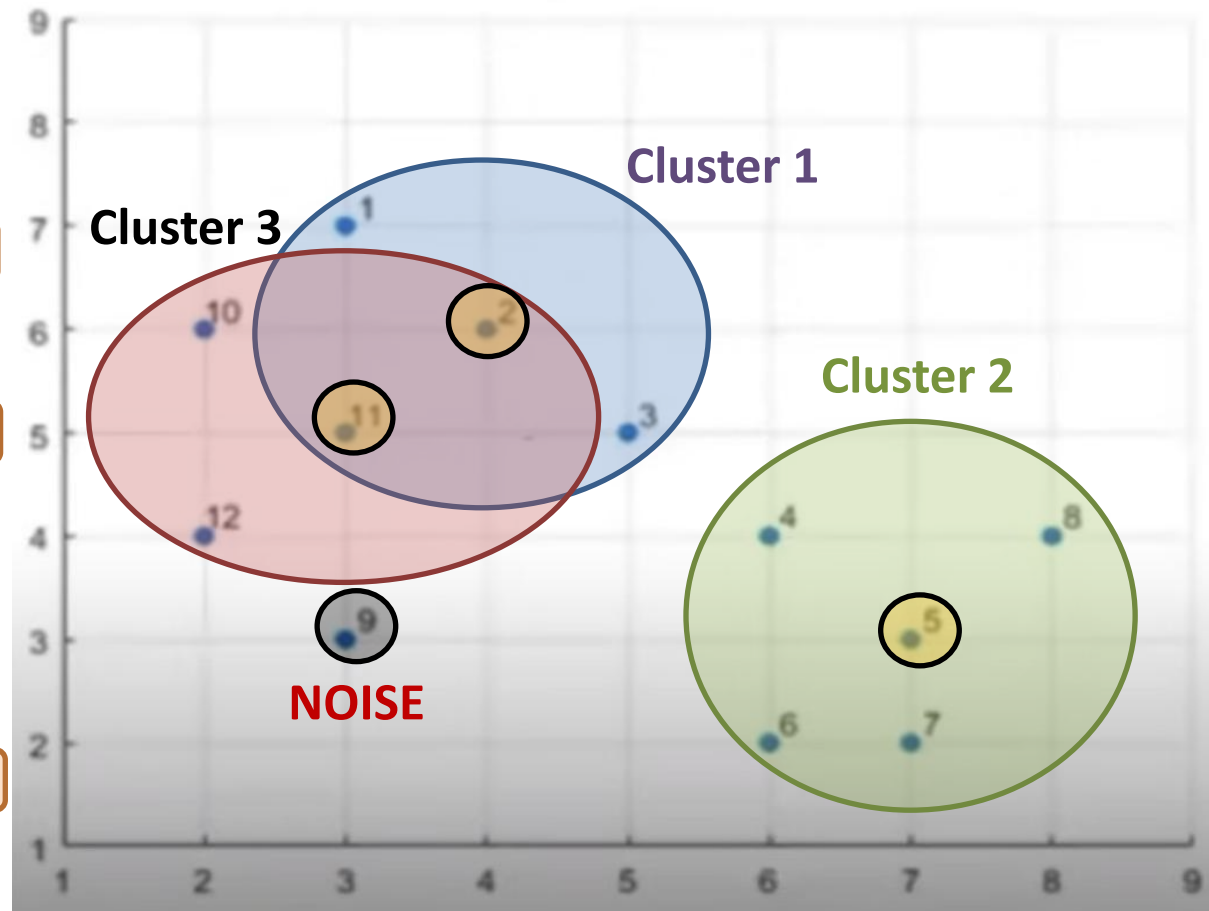
POINT	HORIZONTAL		VERTICAL		
P1			P2	P10	
P2	P1		P3	P11	
P3	P2		P4		
P4	P3		P5		
P5	P4		P6	P7	P8
P6	P5		P7		
P7	P5	P6			
P8	P5				
P9			P12		
P10	P1		P11		
P11	P2	P10	P12		
P12	P9	P11			

POINT	STATUS	
P1	NOISE	BORDER
P2	CORE	
P3	NOISE	BORDER
P4	NOISE	BORDER
P5	CORE	
P6	NOISE	BORDER
P7	NOISE	BORDER
P8	NOISE	BORDER
P9	NOISE	
P10	NOISE	BORDER
P11	CORE	
P12	NOISE	BORDER

DBSCAN ALGORITHM

- **STEP-4:** Plot the clusters based on the CORE data points.

POINT	HORIZONTAL		VERTICAL		
P1			P2	P10	
P2	P1		P3	P11	
P3	P2		P4		
P4	P3		P5		
P5	P4		P6	P7	P8
P6	P5		P7		
P7	P5	P6			
P8	P5				
P9			P12		
P10	P1		P11		
P11	P2	P10	P12		
P12	P9	P11			



DBSCAN ALGORITHM

EXAMPLE TO SOLVE

DBSCAN ALGORITHM

- Apply the DBSCAN algorithm to the given data points.
- Create the clusters with **Minpts = 3** and **epsilon (€) = 1.5**

POINT	COORDINATES	
P1	4	8
P2	5	7
P3	6	6
P4	7	5
P5	8	3
P6	7	3
P7	8	3
P8	9	5
P9	4	4
P10	3	7
P11	4	6
P12	3	5

DIMENSIONALITY REDUCTION

DIMENSIONALITY REDUCTION

- DIMENSIONALITY REDUCTION
- *The number of input features, variables, or columns present in a given dataset is known as dimensionality, and the process to reduce these features is called dimensionality reduction.*

DIMENSIONALITY REDUCTION

- **INTRODUCTION**

- A dataset contains a **huge number of input features** in various cases, which makes the **predictive modeling** task more **complicated**.
- Because it is very difficult to **visualize** or make **predictions** for the **training dataset** with a **high number of features**, for such cases, **dimensionality reduction techniques** are required to use.

DIMENSIONALITY REDUCTION

- **INTRODUCTION**

- Dimensionality reduction technique can be defined as, "***It is a way of converting the higher dimensions dataset into lesser dimensions dataset ensuring that it provides similar information.***"
- It is commonly used in the fields that deal with **high-dimensional data**, such as **speech recognition, signal processing, bioinformatics**, etc. It can also be used for **data visualization, noise reduction, cluster analysis**, etc.

DIMENSIONALITY REDUCTION

- **TYPES OF DIMENSIONALITY REDUCTION**
- **FEATURE SELECTION**
- In this method, we are interested in finding **k** of the total of **n** **features** that give us the most information and we **discard** the other **(n-k) dimensions**.
- Example - Suppose we have **10 features** (**$n=10$**) provided for a **dataset** but we need only **7 features** (**$k=7$**), then we will **discard** the rest **3 features** (**$n-k = 10-7 = 3$**).

DIMENSIONALITY REDUCTION

- **TYPES OF DIMENSIONALITY REDUCTION**
- **FEATURE EXTRACTION**
- In this method, we are interested in finding a **new set of k features** that are combination of the original **n features**.
- There are 2 types of feature extraction techniques
- 1. **Principal Component Analysis (PCA)**
- 2. **Linear Discriminant Analysis (LDA)**

PRINCIPAL COMPONENT ANALYSIS (PCA)

PRINCIPAL COMPONENT ANALYSIS (PCA)

- **INTRODUCTION**
- **Principal Component Analysis** is an **unsupervised learning algorithm** that is used for the **dimensionality reduction** in machine learning.
- It is a **statistical process** that **converts the observations of correlated features** into a set of **linearly uncorrelated features** with the help of **orthogonal transformation**.
- These **new transformed features** are called the **Principal Components**.

PRINCIPAL COMPONENT ANALYSIS (PCA)

- **COMMON TERMS IN PCA**
- **Dimensionality:** It is the number of **features** or **variables** present in the given **dataset**. More easily, it is the number of **columns** present in the **dataset**.
- **Correlation:** It signifies that how **strongly two variables** are **related** to each other. Such as if **one changes**, the other **variable** also gets **changed**.

PRINCIPAL COMPONENT ANALYSIS (PCA)

- **COMMON TERMS IN PCA**
- **Orthogonal:** It defines that **variables** are **not correlated** to each other, and hence the **correlation between** the pair of **variables** is **zero**.
- **Eigenvectors:** If there is a **square matrix M** , and a **non-zero vector v** is given. Then **v** will be **eigenvector** if **Av** is the **scalar multiple** of **v** .
- **Covariance Matrix:** A **matrix** containing the **covariance** between the **pair of variables** is called the **Covariance Matrix**.

PRINCIPAL COMPONENT ANALYSIS (PCA)

- **WORKING OF PCA**
- **Getting the dataset:** Firstly, we need to take the **input dataset** and **divide** it into **two subparts X** and **Y**, where **X** is the **training set**, and **Y** is the **validation set**.
- **Representing data into a structure:** Now we will represent our **dataset** into a **structure**. Such as we will represent the **two-dimensional matrix** of **independent variable X**. Here each **row** corresponds to the **data items**, and the **column** corresponds to the **Features**.

PRINCIPAL COMPONENT ANALYSIS (PCA)

- **WORKING OF PCA**
- **Standardizing the data:** In this step, we will standardize our dataset. Such as in a particular column, *the features with high variance are more important compared to the features with lower variance.*
- If the importance of **features** is **independent** of the **variance** of the **feature**, then we will divide each data item in a **column** with the **standard deviation** of the **column**. Here we will name the **matrix** as **Z**.

PRINCIPAL COMPONENT ANALYSIS (PCA)

- **WORKING OF PCA**
- **Calculating the Covariance of Z:** To calculate the **covariance** of **Z**, we will take the **matrix Z**, and will **transpose** it. After **transpose**, we will **multiply** it by **Z**. The **output matrix** will be the **Covariance matrix** of **Z**.
- **Calculating the Eigen Values and Eigen Vectors:** Now we need to calculate the **eigenvalues** and **eigenvectors** for the **resultant covariance matrix Z**. **Eigenvectors** or the **covariance matrix** are the directions of the **axes** with **high information**. And the **coefficients** of these **eigenvectors** are defined as the **eigenvalues**.

PRINCIPAL COMPONENT ANALYSIS (PCA)

- **WORKING OF PCA**
- **Sorting the Eigen Vectors:** In this step, we will take all the **eigenvalues** and will **sort** them in **decreasing order**, which means from **largest** to **smallest**.
- And simultaneously **sort** the **eigenvectors** accordingly in **matrix P** of **eigenvalues**. The **resultant matrix** will be named as **P***.

PRINCIPAL COMPONENT ANALYSIS (PCA)

- **WORKING OF PCA**
- **Calculating the new features Or Principal Components:** Here we will calculate the **new features**.
- To do this, we will **multiply** the **P^* matrix** to the **Z**.
- In the **resultant matrix Z^*** , each **observation** is the **linear combination** of **original features**.
- Each **column** of the **Z^* matrix** is **independent** of each other.

PRINCIPAL COMPONENT ANALYSIS (PCA)

- **WORKING OF PCA**

- **Remove less or unimportant features from the new dataset:**

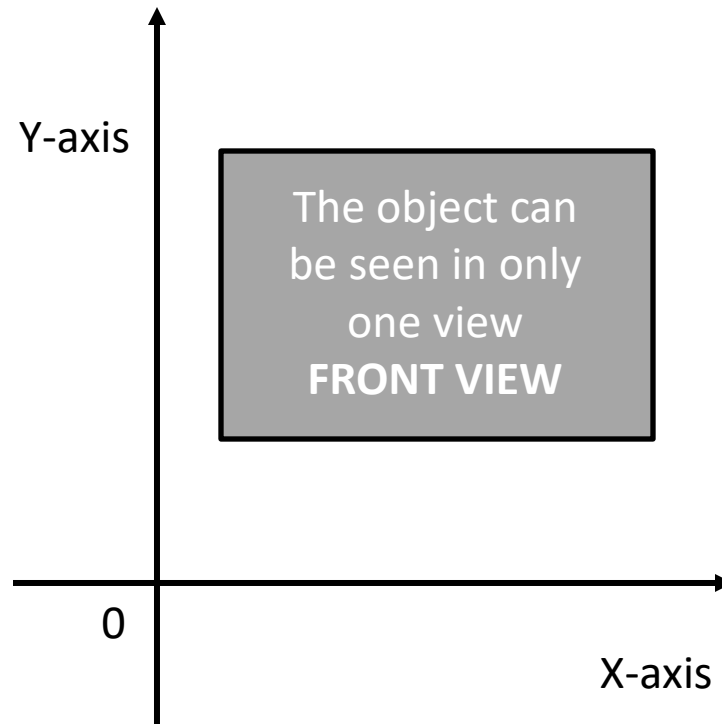
The new feature set has occurred, so we will decide here what to **keep** and what to **remove**.

- It means, we will only keep the **relevant** or **important features** in the **new dataset**, and **unimportant features** will be **removed out**.

PRINCIPAL COMPONENT ANALYSIS (PCA) SOLVED EXAMPLES

PRINCIPAL COMPONENT ANALYSIS (PCA)

- **2D TRANSFORMATIONS [2 DIMENSIONAL]**
- 2D name is given so because it has two axis X and Y.



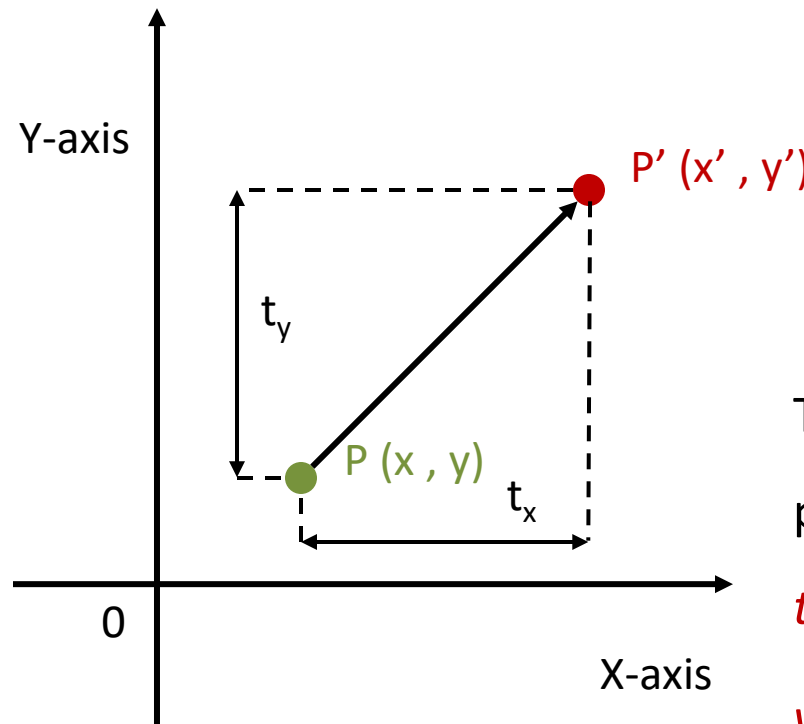
PRINCIPAL COMPONENT ANALYSIS (PCA)

- **TYPES OF 2D TRANSFORMATION**
- It has 3 main types as mentioned below.
- **1. TRANSLATION**
- **2. SCALING**

PRINCIPAL COMPONENT ANALYSIS (PCA)

- **1. TRANSLATION [T]**

- *It is a process of changing the position of object in a straight-line path from one coordinate location to another*



$$x' = x + t_x$$

$$y' = y + t_y$$

The translation distance pair (t_x, t_y) is called as *translation vector* or *shift vector*

PRINCIPAL COMPONENT ANALYSIS (PCA)

- The above translation equations can be represented in single matrix form.

$$\mathbf{P}' = \mathbf{P} + \mathbf{T}$$

$$P = \begin{bmatrix} x \\ y \end{bmatrix}$$

$$P' = \begin{bmatrix} x' \\ y' \end{bmatrix}$$

$$T = \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$

PRINCIPAL COMPONENT ANALYSIS (PCA)

- Example – Translate a polygon with co-ordinates A(2,5), B(7,10) and C(10,2) by 3 units in x direction and 4 units in y direction.

$$\mathbf{P}' = \mathbf{P} + \mathbf{T}$$

$$\mathbf{P} = \begin{bmatrix} x \\ y \end{bmatrix}$$

$$\mathbf{T} = \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$

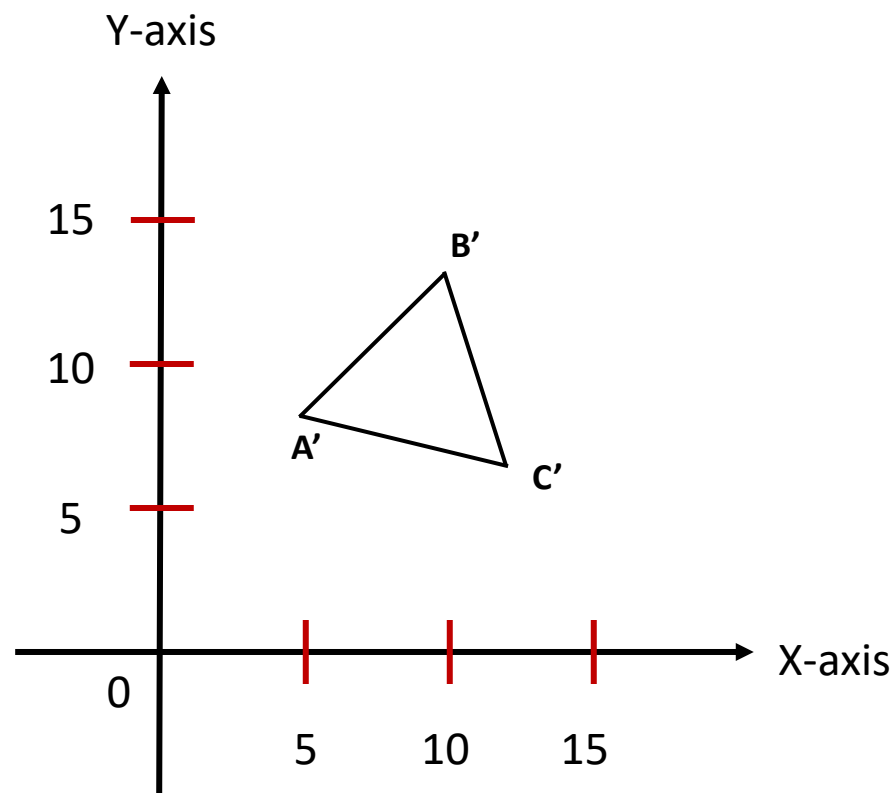
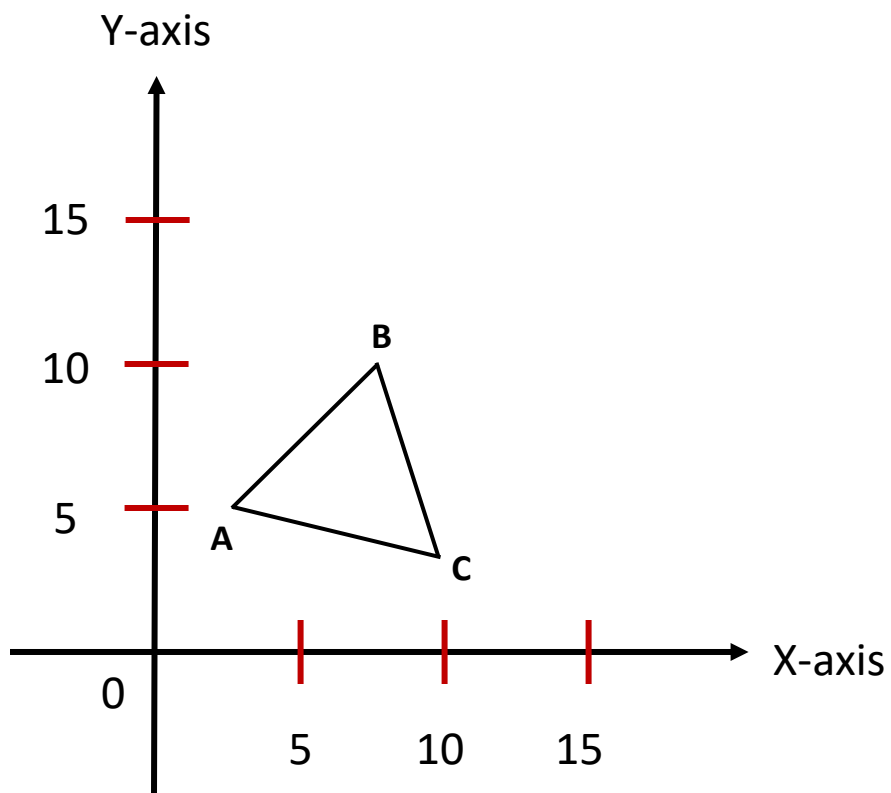
$$\mathbf{A}' = \mathbf{A} + \mathbf{T} = \begin{bmatrix} 2 \\ 5 \end{bmatrix} + \begin{bmatrix} 3 \\ 4 \end{bmatrix} = \begin{bmatrix} 5 \\ 9 \end{bmatrix}$$

$$\mathbf{B}' = \mathbf{B} + \mathbf{T} = \begin{bmatrix} 7 \\ 10 \end{bmatrix} + \begin{bmatrix} 3 \\ 4 \end{bmatrix} = \begin{bmatrix} 10 \\ 14 \end{bmatrix}$$

$$\mathbf{C}' = \mathbf{C} + \mathbf{T} = \begin{bmatrix} 10 \\ 2 \end{bmatrix} + \begin{bmatrix} 3 \\ 4 \end{bmatrix} = \begin{bmatrix} 13 \\ 6 \end{bmatrix}$$

PRINCIPAL COMPONENT ANALYSIS (PCA)

- **OUTPUT**



PRINCIPAL COMPONENT ANALYSIS (PCA)

- **2. SCALING [S]**
- This transformation changes the size of an object.
- This transformation can be carried out for polygons by **multiplying** the coordinate values (x , y) of each vertex by scaling factors S_x and S_y to produce the transformed coordinates (x' , y').

$$x' = x * S_x \qquad y' = y * S_y$$

PRINCIPAL COMPONENT ANALYSIS (PCA)

- The above scaling equations can be represented in single matrix form.

$$\mathbf{P}' = \mathbf{P} \mathbf{S}$$

$$P = \begin{bmatrix} x \\ y \end{bmatrix}$$

$$P' = \begin{bmatrix} x' \\ y' \end{bmatrix}$$

$$S = \begin{bmatrix} S_x & 0 \\ 0 & S_y \end{bmatrix}$$

PRINCIPAL COMPONENT ANALYSIS (PCA)

- **CONDITIONS FOR SCALING**
- Any positive numeric values are valid for scaling factors S_x and S_y
- Values less than 1 reduce the size of the object.
- Values greater than 1 enlarge the size of the object.
- For both S_x and S_y if the value is 1 then the size of the object remains same.
- To get uniform scaling the values of S_x and S_y should be equal.

PRINCIPAL COMPONENT ANALYSIS (PCA)

- Example – Scale the polygon with co-ordinates A(2,5), B(7,10) and C(10,2) by 2 units in x direction and 2 units in y direction.

$$P' = P \times S$$

$$P = \begin{bmatrix} x \\ y \end{bmatrix} \quad P' = \begin{bmatrix} x' \\ y' \end{bmatrix}$$

$$A' = A * S = \begin{bmatrix} 2 & 5 \\ 7 & 10 \\ 10 & 2 \end{bmatrix} \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix} = \begin{bmatrix} 4 & 10 \\ 14 & 20 \\ 20 & 4 \end{bmatrix}$$

$$S = \begin{bmatrix} S_x & 0 \\ 0 & S_y \end{bmatrix}$$

PRINCIPAL COMPONENT ANALYSIS (PCA) EXAMPLE TO SOLVE

PRINCIPAL COMPONENT ANALYSIS (PCA)

- **Translation** – Translate a polygon with co-ordinates $A(4,6)$, $B(8,12)$ and $C(12,4)$ by 4 units in x direction and 5 units in y direction.
- **Scaling** – Scale the polygon with co-ordinates $A(4,6)$, $B(8,12)$ and $C(12,4)$ by 3 units in x direction and 3 units in y direction.

LINEAR DISCRIMINANT ANALYSIS (LDA)

LINEAR DISCRIMINANT ANALYSIS (LDA)

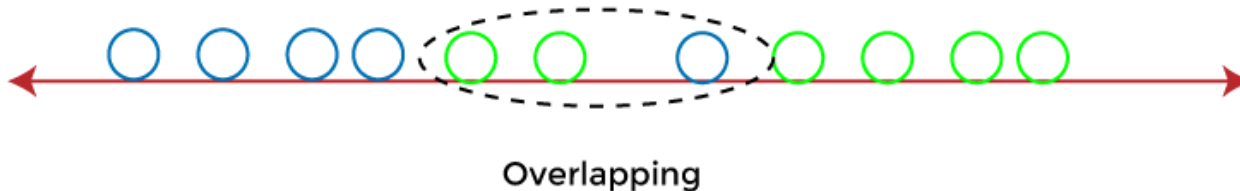
- *Linear Discriminant Analysis (LDA) is one of the commonly used dimensionality reduction techniques in machine learning to solve more than two-class classification problems.*

LINEAR DISCRIMINANT ANALYSIS (LDA)

- **What is Linear Discriminant Analysis (LDA)?**
- **Linear Discriminant analysis** is one of the most popular dimensionality reduction techniques used for supervised classification problems in machine learning.
- It is also considered a **pre-processing step** for modeling differences in **ML** and **applications of pattern classification**.

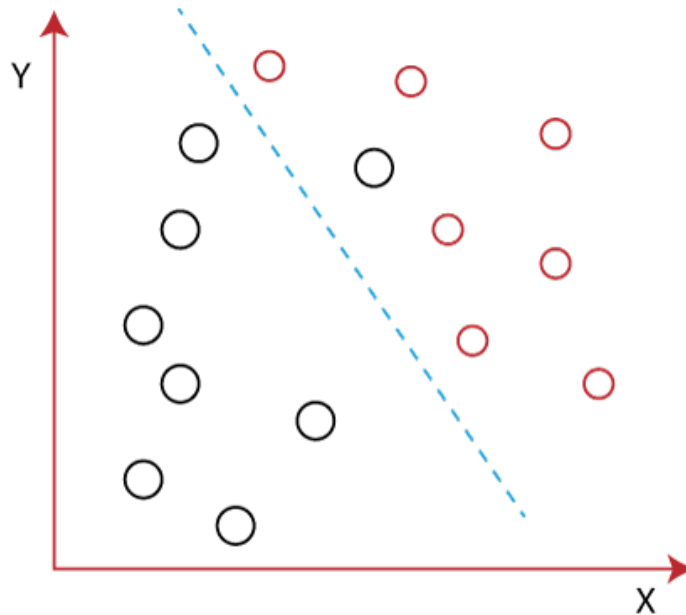
LINEAR DISCRIMINANT ANALYSIS (LDA)

- What is Linear Discriminant Analysis (LDA)?
- For e.g., if we have **two classes** with **multiple features** and need to separate them efficiently.
- When we classify them using a single feature, then it may show **overlapping**.
- To overcome the overlapping issue in the classification process, we must increase the number of **features** regularly.



LINEAR DISCRIMINANT ANALYSIS (LDA)

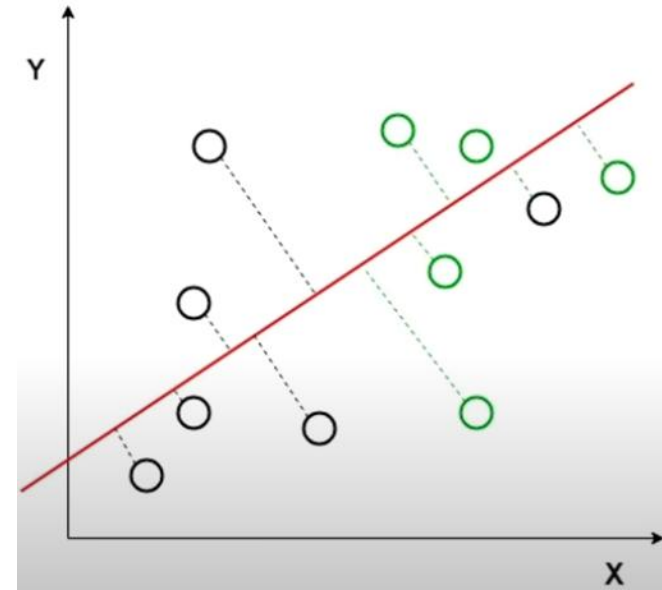
- **EXAMPLE**
- Let's assume we have to classify two different classes having two sets of data points in a 2-dimensional plane as shown below image:



LINEAR DISCRIMINANT ANALYSIS (LDA)

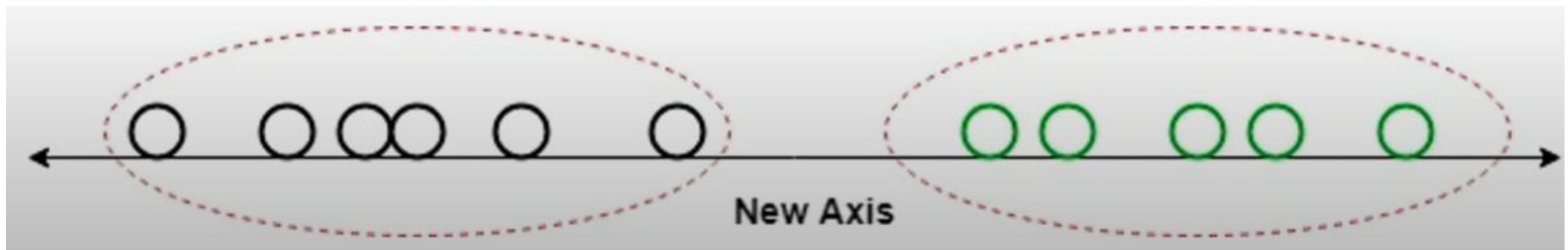
- **EXAMPLE**

- Here, Linear Discriminant Analysis uses both the axes (X and Y) to create a new axis and projects data onto a new axis in a way to maximize the separation of the two categories and hence, reducing the 2D graph into a 1D graph.



LINEAR DISCRIMINANT ANALYSIS (LDA)

- To create a new axis, Linear Discriminant Analysis uses the following criteria:
- It maximizes the distance between means of two classes.
- It minimizes the variance within the individual class.



ANOMOLY DETECTION

ANOMOLY DETECTION

- *Anomaly Detection is the technique of identifying rare events or observations which can raise suspicions by being statistically different from the rest of the observations.*
- Such “**anomalous**” behavior typically translates to some kind of a problem like a **credit card fraud**, **failing machine** in a **server**, a **cyber attack**, etc.

ANOMOLY DETECTION

- An anomaly can be broadly categorized into three categories –
- **Point Anomaly:** A tuple in a dataset is said to be a Point Anomaly if it is far off from the rest of the data.
- **Contextual Anomaly:** An observation is a Contextual Anomaly if it is an anomaly because of the context of the observation.
- **Collective Anomaly:** A set of data instances help in finding an anomaly.

ISOLATION FOREST

ISOLATION FOREST

- **Isolation Forest** is an **unsupervised anomaly detection algorithm** particularly effective for **high-dimensional data**.
- It operates under the principle that **anomalies** are **rare** and **distinct**, making them easier to **isolate** from the **rest of the data**.
- Unlike other methods that profile normal data, Isolation Forests focus on **isolating anomalies**.
- At its core, the Isolation Forest algorithm, it banks on the fundamental concept that **anomalies**, they **deviate significantly**, thereby making them **easier** to **identify**.

ISOLATION FOREST

- The workings of isolation forests are defined below:
- **Building Isolation Trees:** The algorithm starts by creating a set of isolation trees, typically hundreds or even thousands of them.
- These trees are similar to traditional decision trees, but with a key difference: **they are not built to classify data points into specific categories.**
- Instead, isolation trees aim to **isolate individual data points** by **repeatedly splitting** the data based on **randomly** chosen **features** and **split values**.

ISOLATION FOREST

- **Splitting on Random Features:** Isolation trees introduce **randomness** at each **node** of the tree, a **random feature** from the **dataset** is selected.
- Then, a **random split value** is chosen within the **range** of that **particular feature's values**.
- This **randomness** helps ensure that **anomalies**, which tend to be **distinct** from the **majority** of **data points**, are **not hidden** within **specific branches** of the **tree**.

ISOLATION FOREST

- **Isolating Data Points:** The **data points** are then **directed down the branches** of the **isolation tree** based on their **feature values**.
 - If a **data point's value** for the chosen **feature** falls below the **split value**, it goes to the **left branch**. Otherwise, it goes to the **right branch**.
 - This process continues **recursively** until the **data point** reaches a **leaf node**, which simply represents the **isolated data point**.

ISOLATION FOREST

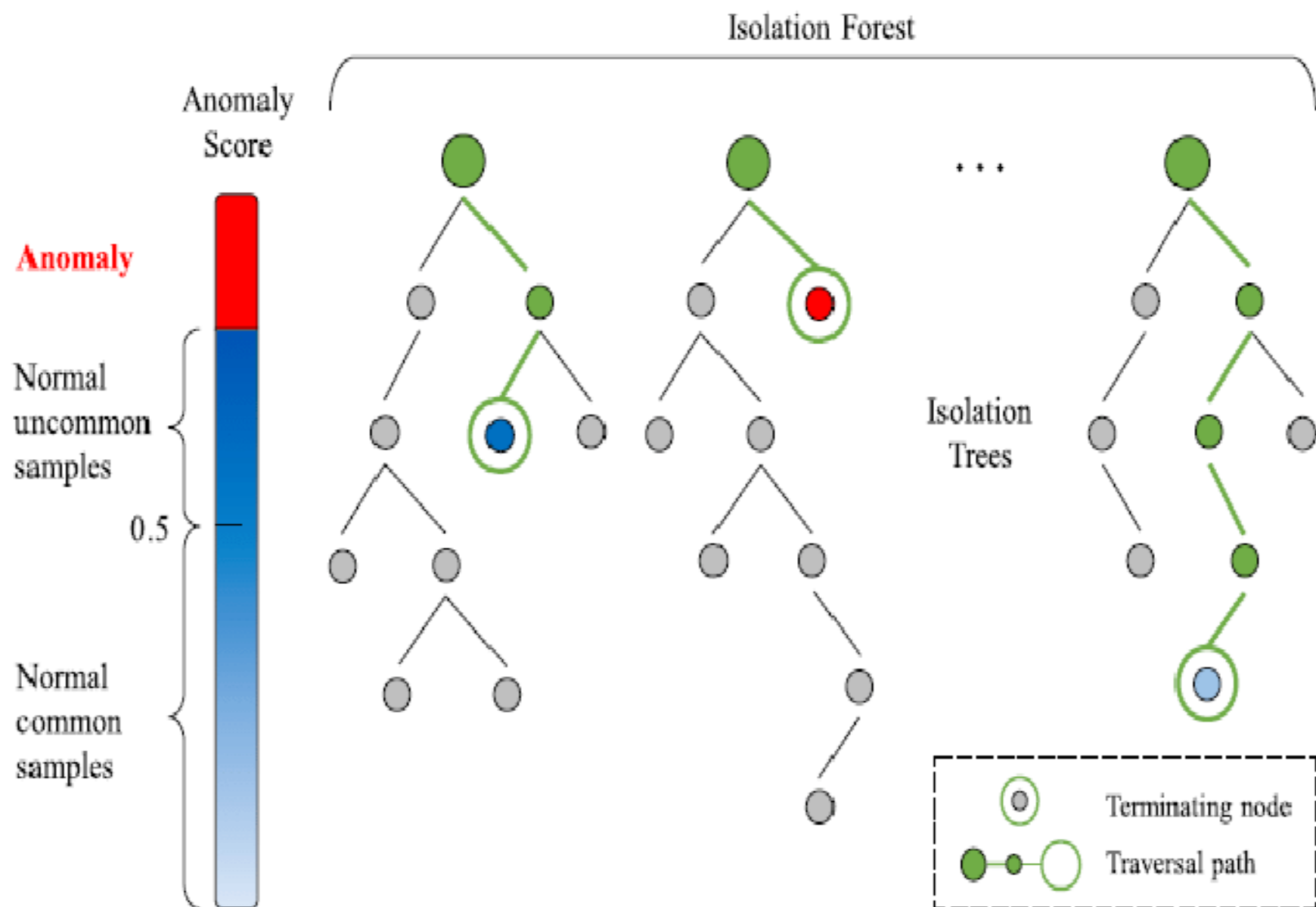
- **Anomaly Score:** The key concept behind Isolation Forests lies in the path length of a data point through an isolation tree.
 - Anomalies, by virtue of being different from the majority, tend to be easier to isolate. They require fewer random splits to reach a leaf node because they are likely to fall outside the typical range of values for the chosen features.
 - Conversely, normal data points, which share more similarities with each other, might require more splits on their path down the tree before they are isolated.

ISOLATION FOREST

- **Anomaly Score Calculation:** Each data point is evaluated through all the isolation trees in the forest.
 - For each tree, the **path length** (number of splits) required to **isolate** the **data point** is **recorded**.
 - An **anomaly score** is then **calculated** for **each data point** by **averaging** the **path lengths** across all the **isolation trees** in the **forest**.

ISOLATION FOREST

- **Identifying Anomalies:** Data points with shorter average path lengths are considered more likely to be anomalies.
- This is because they were easier to isolate, suggesting they deviate significantly from the bulk of the data.
- A threshold is set to define the anomaly score that separates normal data points from anomalies.



ISOLATION FOREST ANOMOLY DETECTION METHODS

ISOLATION FOREST

- **OUTLIER (Anomaly)**
- These are the **values** in **dataset** which **standout** from the **rest** of the **data**.
- It can be a **result** of **error** in **reading**, **fault** in the **system**, **manual error** or **misleading**.
- **1. Interquartile Range (IQR)**
- **2. Z- Score**

IQR SCORE ANOMOLY DETECTION METHOD

ISOLATION FOREST

- **IQR EXAMPLE**

- Consider below dataset of 15 points

Example: [-50, -2, 8, 19, 29, 33, 34, 49, 50, 53, 59, 64, 87, 99, 1456]

- **STEP – 1** : Arrange the *data(n)* in **ascending** order.

[-50, -2, 8, 19, 29, 33, 34, 49, 50, 53, 59, 64, 87, 99, 1456]

ISOLATION FOREST

- **IQR EXAMPLE**

[-50, -2, 8, 19, 29, 33, 34, 49, 50, 53, 59, 64, 87, 99, 1456]

- **STEP – 2** : Calculate the quartiles and IQR

$$Q1: \frac{1*(N+1)^{th}}{4} \text{ value} \rightarrow 4^{th} \text{ Value} \rightarrow 19$$

$$Q3: \frac{3*(N+1)^{th}}{4} \text{ value} \rightarrow 12^{th} \text{ Value} \rightarrow 64$$

$$IQR: Q_3 - Q_1 = 64 - 19 = 45$$

ISOLATION FOREST

- **IQR EXAMPLE**

`[-50, -2, 8, 19, 29, 33, 34, 49, 50, 53, 59, 64, 87, 99, 1456]`

- **STEP – 3 : Define threshold**

$$\textit{Threshold} = 1.5 * \textit{IQR} = 1.5 * 45 = 67.5$$

ISOLATION FOREST

- **IQR EXAMPLE**
- **STEP – 4** : Identify outliers
- Any point **less than -Q1(-19) threshold** or **greater than +Q3(+64) threshold** is considered an **outlier**

$$\text{Lower threshold: } 19 - 67.5 = -48.5$$

$$\text{Upper threshold: } 64 + 67.5 = 131.5$$

-50, -2, 8, 19, 29, 33, 34, 49, 50, 53, 59, 64, 87, 99, 1456]

- **We will remove the first and last point in this dataset (-50 & 1456)**

Z-SCORE ANOMOLY DETECTION METHOD

ISOLATION FOREST

- **Z SCORE EXAMPLE**

- Consider the dataset of daily sales revenue of 30 days

[100, 150, 120, 125, 140, 130, 110, 135, 130, 150, 140, 100, 95, 80, 120, 125, 130,
100, 140, 135, 130, 145, 110, 120, 130, 135, 140, 125, 130, 120]

- We need to identify any days where the sales revenue is significantly different from the other days, which may indicate an anomaly or outlier.

ISOLATION FOREST

- **Z SCORE EXAMPLE**

- **STEP-1:** Calculate mean and standard deviation.

[100, 150, 120, 125, 140, 130, 110, 135, 130, 150, 140, 100, 95, 80, 120, 125, 130,
100, 140, 135, 130, 145, 110, 120, 130, 135, 140, 125, 130, 120]

$$\text{Mean: } \mu = \frac{\sum_{i=1}^{30} x_i}{n} = \frac{100+150+120+\dots+130+120}{30} = 123.5$$

Standard deviation:

$$\sigma = \sqrt{\frac{\sum_{i=1}^{30} (x_i - \mu)^2}{n-1}} = \sqrt{\frac{(100-123.5)^2 + (150-123.5)^2 + \dots + (120-123.5)^2}{30-1}} = 20.0$$

ISOLATION FOREST

- **Z SCORE EXAMPLE**
- **STEP-2:** Calculate z-scores.

[100, 150, 120, 125, 140, 130, 110, 135, 130, 150, 140, 100, 95, 80, 120, 125, 130,
100, 140, 135, 130, 145, 110, 120, 130, 135, 140, 125, 130, 120]

$$Z = \frac{x - \mu}{\sigma}$$

$$Z_1 = \frac{100 - 123.5}{20} = -1.17$$

[-1.17, 0.14, -0.28, -0.16, 0.56, 0.08, -0.89, 0.32, 0.08, 0.14, 0.56, -1.17, -1.45, -
2.22, -0.28, -0.16, 0.08, -1.17, 0.56, 0.32, 0.08, 0.86, -0.89, -0.28, 0.08, 0.32, 0.56, -
0.16, 0.08, -0.28]

ISOLATION FOREST

- **Z SCORE EXAMPLE**
- **STEP-3:** Set a threshold.

We set a threshold for what we consider an anomaly.

Usually z-score =3 is considered as a cut-off value to set the limit.
which captures 99.7% of the data in a normal distribution

Therefore, any z-score greater than +3 or less than -3 is considered as outlier which is pretty much similar to standard deviation method.

ISOLATION FOREST

- **Z SCORE EXAMPLE**
- **STEP-4:** Identify anomalies.

[-1.17, 0.14, -0.28, -0.16, 0.56, 0.08, -0.89, 0.32, 0.08, 0.14, 0.56, -1.17, -1.45, -2.22, -0.28, -0.16, 0.08, -1.17, 0.56, 0.32, 0.08, 0.86, -0.89, -0.28, 0.08, 0.32, 0.56, -0.16, 0.08, -0.28]

- No outliers found in the given data.

IQR & Z-SCORE ANOMOLY DETECTION EXAMPLES

ISOLATION FOREST

- **IQR EXAMPLE**

- Consider following dataset and find out the outliers
- [10,20,15,-30,22,60,45,-24,55,81]

- **Z SCORE EXAMPLE**

- Consider following dataset of daily transaction and find out the outliers. Consider the Z score = 5
- [100,150,120,200,250,101,230,330,450,500]